

#-----

1. problem solving
2. logical and maintainable code
3. algorithms and data structures
4. ML function
5. ML system design

周二 最后 bar raiser

October 20, 12:00pm - 1:00pm, Apoorv Gupta

1. ML functional
- 2 LP, bias for action, insist on the highest standards

can I hands on ML code capability, write a training loop, forward backward, define loss, then ask more details, what pytorch does

modeling tech, grid clipp, MP precision, autocast, where to use fp32, use bf 16
training gramework,
autograd,
torch eager mode, dynamo eager mode, aten operator, autograd,
model training process, training used techniques
from the training loop, how to eval, how to select data, if I want to train, how to log, how to
log metrics value, how to save checkpoints
distributed training, using torch fsdp, megatron
how to model design, implement a model, how to implement a embedding model, product
embedding, loss function, cross entrophy
if layernorm, rmsnorm implementation
separate tensor, how to

function more on breadth

2.

October 20

2:00pm - 3:00pm

Amith Mamidala

ML system design

LP: earn trust, think big

system related:

ML system, compute roof lining, performance, memory, compute
pattern:

- 1) given a model, estimate memory consumption, parameters, how much memory, if activation uses a lot memory, calculate it
- 2) if the model could fit, training throughput, how to increase throughput,
--> change local batch size, gradient accumulation
- 3) if model can not fir, how to separate the model, using tp, fsdp, ep, sp
after separation

MFU, using model flops to calculate hardware training utilization, different hardware has peaks, depends on training performance, how to use the flops
calculate each layer model flops, MLP and attention

4) how to separate the model, asking each separation, how the collective communication, using all gather, then use it to do gradients

5) adding the communication cost to MFU, estimate the impact on training performance

6) what are the trade off, how to make decision, why do this type of separation

tp degree

activation checkpoint??? selectively choose activation checkpoint recompute, or use tp 4 to divide the mode, which one to use

1) recompute, or 2) using tp

fsdp can not separate activation

end to end work through, analyze the training resources,

key interview!!!

given qwen,

each layer, check point

using tp, how to separate

whether to use sp

mention in LP

kernel optimization, algo, compute, memory optimization

kernel function

3.

October 20

5:00pm - 6:00pm

Hanbo Wang

coding, data structure and algorithms

LP: 1. have backbone, disagree and commit, 2. ownership

leetcode

big O(n)

1) sub optimization sol, then optimize

2) optimal sol, first mention brute force

4.

October 21

11:00am - 12:00pm

Rui Silva

coding, problem solving

LP: 1. customer obsession, 2. invent and simplify

focus on how to use coding to solve practical problems, related to leetcode

5.

October 21
12:00pm - 1:00pm
Sanjay Bhatia

coding: logical and maintainable
coding solution break into structures, modules, if need more components, how easily to add new function
OO design?

LP: 1. deliver results, 2. dive deep

focus on: non tech aspect, tech leading experience, how to lead the team to finish the project as a L6, visionary, give the team tech direction, how to build the team's tech direction
how to think big, how to give long term tech direction

not only focus on deliver result, but also on tech lead,

end to end optimization
analyze model structure, how to improve model performance
distributed + non distributed

kernel or compiler
operator fuse

####-----

####-----

Ads LP not good:

behavior question 题库

1.
ownership LP: bad
provide short term benefits, do not mention sacrifice long term benefits
make short term success rather than long term value, I didn't mention optimize
also if I evaluate

describe the setbacks or problems without talking about how to address them
I mentioned a problem, without an investigation, and how to solve it

rely on other to remove the setbacks,

2.
have backbone, disagree and commit
instead of using strategic preparation, I shouldn't do the poc demo by myself, I should consent with my manager

3.
deliver results:
failed to address valid brand perception concerns, could have explore more nuance in model,

balance revenue optimization with company impression, before pushing the model through

4. if using coupang cheap model example,
do not mentioned long term impression to happens, it is a risk, and how big the risk, how long it could be, how to remove it
how to evaluate it

5. science
can I describe concept in structed way,
knowledge in model details, loss function in clip model, didn't mention contrastive

6. follow up: online a/b test, better engagement, found customer historical spend,

####-----
####-----
amazon Annapurna Lab
####-----
####-----

BIAS FOR ACTION

Definition and Indicators

Bias for Action

Speed matters in business. Many decisions and actions are reversible and do not need extensive study. We value calculated risk taking.

Concern	Mild Concern	Mixed	Mild Strength	Strength
• Moves slower than necessary when faced with incomplete information • Hesitates longer than necessary when making even small decisions • Fears making mistakes and doing things that haven't been done before			• Moves quickly on a project, even when some information is lacking • Understands when to consult with others and when it's not necessary • Willing to make small progress toward a solution rather than finding the perfect solution right away • Deals with problems quickly so others can make progress • Carefully considers what is important to get done right away and what can wait	

Bias for Action - Suggested Behavioral Interview Questions

Please choose 2-3 questions below to prepare for your interview.

BIAS FOR ACTION

Definition and Indicators				
Bias for Action				
<i>Speed matters in business. Many decisions and actions are reversible and do not need extensive study. We value calculated risk taking.</i>				
Concern	Mild Concern	Mixed	Mild Strength	Strength
- Moves slower than necessary when faced with incomplete information - Hesitates longer than necessary when making even small decisions - Fears making mistakes and doing things that haven't been done before			- Moves quickly on a project, even when some information is lacking - Understands when to consult with others and when it's not necessary - Willing to make small progress toward a solution rather than finding the perfect solution right away - Deals with problems quickly so others can make progress - Carefully considers what is important to get done right away and what can wait	

Bias for Action - Suggested Behavioral Interview Questions

Please choose 2-3 questions below to prepare for your interview.

- Give me an example of a calculated risk that you have taken where speed was critical.** What was the situation and how did you handle it? What steps did you take to mitigate the risk? What was the outcome? Knowing what you know now, would you have done anything differently?
- Tell me about a time when you worked against tight deadlines and didn't have time to consider all options before making a decision.** How much time did you have? What approach did you take? What did you learn from the situation?
- Describe a situation where you made an important business decision without consulting your manager.** What was the situation and how did it turn out? Would you have done anything differently?
- Tell me about a time when you had to gather information and respond immediately to a situation.** What was the outcome? Would you have done anything differently?
- Give me an example of when you had to make an important decision and had to decide between moving forward or gathering more information.** What did you do? What was the outcome? What information is necessary for you to have before acting?
- Tell me about a time when you saw an issue that would impact your team and took a proactive approach to solve it.** What was the issue? What did you do and what was the outcome? What did you learn from this situation?
- Tell me about a time when you felt your team was not moving to action quickly enough.** What was the situation? What did you do? What was the outcome? Would you have done anything differently? (Manager)
- Tell me about a time when you were able to remove a serious roadblock/barrier preventing your team from making progress.** What was the barrier? How were you able to remove it? What was the outcome? Knowing what you know now, would you have done anything differently? (Manager)

####-----
LP
bias for action
####-----

amazon Annapurna Lab SDE interview use

1. STAR,
2. 16 amazon leadership principal

#-----

question:

1. Give me an example of a calculated risk that you have taken where speed was critical.
2. Tell me about a time when you worked against tight deadlines and didn't have time to consider all options before making a decision.
3. Describe a situation where you made an important business decision without consulting your manager.
4. Tell me about a time when you had to gather information and respond immediately to a situation.
5. Give me an example of when you had to make an important decision and had to decide between moving forward or gathering more information.
6. Tell me about a time when you saw an issue that would impact your team and took a proactive approach to solve it.
7. Tell me about a time when you felt your team was not moving to action quickly enough.
(Manager)
8. Tell me about a time when you were able to remove a serious roadblock/barrier preventing your team from making progress. (Manager)

#-----

请根据我的简历，使用

1. STAR + feedback, feedback means if I do it next time, how could I do better?

2. 14 amazon rules来举例回答,
3. Situation and Task 1-2 minutes, Action 5-6 minutes, Results 1-2 minutes
4. for action, if possible, please list option A, B, C, and why you choose one option, why you make this decision
5. Compare option A, B, C, and their trade off, Include risk analysis
6. also include backup plan

#-----
bias for action, story 1
#-----

Of course. Crafting a strong answer using the STAR method and Amazon's Leadership Principles is key for this interview. Based on your experience at Coupang, here is a detailed example of how you could answer the question about taking a calculated risk where speed was critical.

This story focuses on the problem you mentioned in your resume: "Solved model performance significantly different between Online A/B test and Offline evaluation."

My Answer Structure

Situation (1-2 minutes): Set the scene at Coupang. The problem was a critical loss of trust in the ML development process, threatening a major product launch.

Task (Part of Situation): My goal was to fix this discrepancy under a tight deadline to unblock a crucial model launch for the holiday season.

Action (5-6 minutes): This is the core of the story. I'll detail my investigation, the options I considered (A, B, C), why I chose my specific path (the calculated risk), and the backup plan I put in place.

Result (1-2 minutes): I'll explain the successful outcome, quantify the business impact, and talk about how it restored trust.

Feedback (Self-Reflection): I'll reflect on what I learned and how I would approach a similar situation in the future by investing in long-term solutions.

Here we go.

Interview Answer Example (S) Situation

"At Coupang, our Search and Ads teams were heavily reliant on offline evaluation and online A/B testing to innovate and ship better machine learning models. We hit a major roadblock where we saw a recurring, severe discrepancy between our offline and online results. For example, a new Deep Learning model would show a fantastic 15% lift in our offline evaluation metrics, but when we shipped it to an online A/B test, the real-world business metrics would be flat, or in some cases, even negative.

This wasn't just a technical problem; it was a business crisis. It eroded the organization's trust in our ML platform, developers were becoming hesitant to innovate, and our velocity for shipping improvements plummeted.

(T) Task

We were less than two months away from the peak holiday shopping season, and we had a state-of-the-art ranking model that we believed could significantly increase revenue. However, we couldn't launch it because we couldn't trust our offline evaluation. My task was to lead the effort to diagnose and resolve this online-offline gap with extreme urgency. I had to find a solution that was fast enough to unblock this critical launch, which meant I couldn't afford a months-long investigation. I had to take a calculated risk. This perfectly aligns with the Amazon principle of Bias for Action; we needed to move quickly despite the ambiguity.

(A) Action

My first step was to Dive Deep. I took Ownership of the entire problem and audited the end-to-end pipeline. This included our data logging from production, the feature generation logic for both offline training and online serving, the offline evaluation framework, and the online A/B testing platform. I quickly identified three potential root causes: data leakage in our offline datasets, a mismatch in user population distributions, and—my strongest hypothesis—an inconsistency in how time-sensitive features were computed between the real-time online serving path and the offline batch-processing path.

Given the tight deadline, a complete, multi-month overhaul was not an option. Speed was the most critical factor. I framed three distinct options for my director and team:

Option A: The Safe, Slow Approach. This involved pausing all major model launches and initiating a comprehensive, ground-up rebuild of our feature logging and generation pipelines. This would guarantee a correct, long-term fix but would take at least four months. The consequence was certain: we would miss the holiday season and the significant revenue opportunity associated with our new model.

Option B: The Calculated Risk & My Recommendation. I proposed we aggressively pursue my primary hypothesis: the feature inconsistency. Instead of rebuilding the whole system, I suggested we build a lightweight, near-real-time "feature parity monitoring" service. This tool would sample production traffic and compare the feature vectors generated for online serving against those generated by our offline pipeline for the exact same events. This wouldn't fix the root cause, but it would quantify the drift. We could then use this drift to correct our offline metrics, giving us a reliable-enough signal to make a launch decision. The risk was that my hypothesis could be wrong, and we'd spend 2-3 critical weeks on a tool that didn't solve the problem. However, the potential reward was unblocking our most important launch of the year.

Option C: The High-Risk Gamble. This was to essentially ignore offline evaluation and use a series of rapid, small-scale online A/B tests to iterate. This was fast but incredibly inefficient and put the user experience at risk. We'd be flying blind and could easily degrade search quality.

I advocated strongly for Option B. It was the most Frugal approach in terms of immediate engineering effort and allowed us to move forward quickly. It was laser-focused on Delivering Results against the most critical business deadline. I convinced leadership that the calculated risk of my hypothesis being wrong was a better bet than the certainty of failure with Option A or the recklessness of Option C.

To mitigate the risk of Option B failing, I also established a backup plan. While my core team focused on the feature parity monitor, I tasked one engineer with a parallel workstream to begin the initial design and scoping for the full pipeline overhaul from Option A. This way, if the

monitor didn't yield a solution within three weeks, we wouldn't be starting the long-term fix from a complete standstill. We would have already done the groundwork.

My team and I built the core monitoring service in just under three weeks.

(R) Result

The calculated risk paid off immediately and decisively. The feature parity monitor pinpointed the issue within days of deployment. It found a 30% value mismatch in a critical user-session feature, which was caused by a subtle caching difference between our online and offline environments.

With this data, we created a correction model for our offline evaluation scores. The holiday ranking model's predicted lift was adjusted from a suspicious +15% to a much more realistic +4%. This gave us the confidence to launch. The online A/B test came back with a 3.8% lift in GMV, almost perfectly matching our corrected prediction.

The business impact was enormous. We successfully launched the model before the holiday peak, which was estimated to have contributed over \$10 million in annualized revenue. Just as importantly, we Earned Trust back from the entire organization. The feature parity monitor became a permanent, mandatory part of our ML launch process.

(Feedback / Learn and Be Curious)

Looking back, while our bias for action was the right call, the situation exposed a foundational weakness. We were forced to be reactive. The calculated risk was necessary because we lacked proactive monitoring. My key learning was that while speed is critical, it must be built on a reliable foundation.

If I were to face that situation again, I would have pushed for this kind of parity monitoring before it became a four-alarm fire. In fact, following this project's success, I championed the long-term investment to build a unified feature store, evolving our quick monitor into a robust platform. This Invented and Simplified our entire workflow by ensuring feature consistency by design, not just by validation. It taught me how to balance the short-term need to deliver results with the long-term vision of building scalable, resilient systems.

####-----
LP
insist on the highest standards
####-----

#-----
insist on the highest standards, story 1
#-----

INSIST ON THE HIGHEST STANDARDS

Definition and Indicators				
Insist on the Highest Standards				
Leaders have relentlessly high standards - many people may think these standards are unreasonably high. Leaders are continually raising the bar and drive their teams to deliver high quality products, services, and processes. Leaders ensure that defects do not get sent down the line and that problems are fixed so they stay fixed.				
Concern	Mild Concern	Mixed	Mild Strength	Strength
- Conducts minimal reviews of others' work - Emphasizes speed over quality for most projects - Fails to recognize the long-term impact of lower standards - Creates or accepts goals that are too easily achievable - Accepts quick solutions to problems with minimal follow-up to ensure they are addressed			- Sets and commits to goals that are challenging, yet realistic - Reviews work extensively and offers high quality feedback - Communicates and gets agreement on expected standards - Builds systems that are scalable and serve customer needs - Continually tries to improve processes	

Insist on the Highest Standards - Suggested Behavioral Interview Questions

Please choose 2-3 questions below to prepare for your interview.

1. Describe a time when you refused to compromise your standards around quality/customer service, etc. Who was your customer? What was the result?
2. Tell me about a time when you were unsatisfied with the status quo. What did you do to change it? What was the impact? Would you do anything differently in the future?
3. Tell me about a time when you worked to improve the quality of a product / service / solution that was already getting good customer feedback. Why did you think it needed improvement? How did customers react?
4. Tell me about a time when you had to make a decision between standards and delivery. What tradeoffs did you have to make? What was the outcome? Knowing what you know now, would you have done anything differently?
5. Give me an example of a goal you've had where you wish you had done better. What was the goal? How could you have improved on it?
6. Describe the most significant continuous improvement project that you led. What was the catalyst to this change and how did you go about it? What was the outcome?
7. Tell me about a time when you used feedback about your team to drive a change. How did you gather or receive feedback on your team's performance? What was the outcome? (Manager)

INSIST ON THE HIGHEST STANDARDS

Definition and Indicators				
Insist on the Highest Standards				
Leaders have relentlessly high standards - many people may think these standards are unreasonably high. Leaders are continually raising the bar and drive their teams to deliver high quality products, services, and processes. Leaders ensure that defects do not get sent down the line and that problems are fixed so they stay fixed.				
Concern	Mild Concern	Mixed	Mild Strength	Strength
- Conducts minimal reviews of others' work - Emphasizes speed over quality for most projects - Fails to recognize the long-term impact of lower standards - Creates or accepts goals that are too easily achievable - Accepts quick solutions to problems with minimal follow-up to ensure they are addressed			- Sets and commits to goals that are challenging, yet realistic - Reviews work extensively and offers high quality feedback - Communicates and gets agreement on expected standards - Builds systems that are scalable and serve customer needs - Continually tries to improve processes	

Insist on the Highest Standards - Suggested Behavioral Interview Questions

Please choose 2-3 questions below to prepare for your interview.

Example: Ensuring Numerical Integrity in LLM Kernel Development

Amazon Leadership Principle in Focus: Insist on the Highest Standards

Leaders have relentlessly high standards — many people may think these standards are unreasonably high. Leaders are continually raising the bar and drive their teams to deliver high quality products, services, and processes. Leaders ensure that defects do not get sent down the line and that problems are fixed so they stay fixed.

(Situation & Task - 1.5 minutes)

"In my recent role as a Software Architect in AI/ML at the Research Institute in Hong Kong, I was leading a project to develop and optimize low-level inference kernels for Large Language Models. Our goal was to improve inference performance by 30% through techniques like quantization. During the validation phase of a new, highly optimized kernel implementing the Duquant and Hadamard transpose algorithms, we encountered a critical but subtle issue: the kernel passed all our standard unit tests, but under specific, high-load conditions, it would sporadically produce a numerically inaccurate output.

The inaccuracy was small—often in the 5th or 6th decimal place—and didn't always cause a visible failure. The initial reaction from some team members was to de-prioritize it, arguing that the performance gains were significant and the error rate was within an acceptable, albeit non-zero, threshold for most applications. However, I recognized this as a fundamental quality defect. In mission-critical domains like autonomous systems, financial analysis, or scientific computing, which are target use cases for our platform, such numerical instability is unacceptable. It could lead to cascading, undetectable errors. My task was to resolve this issue comprehensively, insisting on a standard of absolute numerical integrity, even if it meant delaying the launch."

(Action - 6 minutes)

"I halted the rollout and convened the team. We defined the problem not as a "minor bug," but as a critical failure in our quality gate. I presented three options for a path forward.

Option A: Implement a Software Workaround. Add a post-processing check in the kernel's output to detect and correct for the known numerical deviation. This would be a targeted fix for the specific symptom.

Option B: Patch the Kernel with Robust Error Handling. Keep the core algorithm but surround it with extensive, low-level error checking and recovery mechanisms (e.g., boundary checks, sanity checks on intermediate results) to prevent the memory corruption from causing visible inaccuracies.

Option C: Root Cause Analysis and Fundamental Correction. Launch a deep, investigative effort to find the root cause of the memory corruption, which we suspected was a race condition or a memory alignment issue, and rewrite the kernel to eliminate the defect at its source.

Comparison and Trade-off Analysis:

Option A (Workaround):

Pros: The fastest solution. We could meet the original launch deadline. Minimal engineering effort.

Cons & Risks: This is a classic "papering over the cracks" approach. It only treats the symptom, not the disease. It adds latency and complexity. Most critically, it sets a dangerous precedent that low-level numerical inaccuracies are tolerable. There was a high risk that other, undetected instabilities existed. This violated the principle of ensuring "defects do not get sent down the line."

Why I didn't choose it: It was the antithesis of high standards. It was a compromise on quality for the sake of speed.

Option B (Patch with Error Handling):

Pros: More robust than Option A. It could potentially catch a broader class of errors and prevent them from propagating. It respected the engineering effort already invested in the algorithm.

Cons & Risks: It added significant complexity and performance overhead to our high-performance kernel. The core logic remained flawed and potentially unstable. We would be building a system on a known-weak foundation. The long-term maintenance cost would be high, and the risk of a novel, unhandled error condition persisting was still present.

Why I didn't choose it: While it showed more diligence, it still fell short of the highest standard. It was a defensive patch, not a quality-built core.

Option C (Root Cause Analysis):

Pros: This was the only option that aimed to fix the problem so it stays fixed. It would result in a correct, reliable, and high-performance kernel. It would deepen our team's understanding of the system and prevent similar bugs in the future. It aligned perfectly with "insisting on the highest standards."

Cons & Risks: This was the most time-consuming and resource-intensive path. It required deep expertise in parallel computing and GPU memory architecture. It jeopardized our aggressive performance improvement timeline.

Why I chose this: I argued that shipping a kernel with a known numerical defect was a breach of trust with our customers and a failure of our engineering rigor. The short-term delay was a necessary investment in the long-term reliability and reputation of our platform. This decision embodied *Are Right, A Lot* (trusting our judgment on quality over schedule pressure) and *Ownership* (taking responsibility for the entire product's health).

My Decision and Execution:

I mandated that we pursue Option C. I led the root cause analysis, using advanced profiling tools and formal methods to isolate the issue to a subtle memory alignment problem exacerbated by a race condition during a specific tensor manipulation in the Hadamard transpose. We didn't just fix the bug; we redesigned that section of the kernel to be memory-safe by construction, using better synchronization primitives and aligned memory access patterns.

Backup Plan: I was aware of the schedule risk. My backup plan was that if the root cause analysis exceeded two weeks without a clear path, we would temporarily roll back to the previous, stable (but slower) kernel for the upcoming release. This would protect our customers

from the defect while giving us the necessary time to perfect the new one. We would then communicate a revised timeline to stakeholders, emphasizing our commitment to quality."

(Results - 1.5 minutes)

"The results validated the insistence on high standards. We successfully identified and eradicated the root cause. The new, corrected kernel not only maintained the 30% performance improvement but did so with zero numerical inaccuracy across billions of inference operations in our stress tests.

More importantly, this episode raised the bar for our entire team. We institutionalized the lessons learned by adding new, rigorous stress tests for numerical stability to our continuous integration pipeline. This prevented similar defects from ever making it to the validation stage again. The kernel became the gold standard for reliability within the organization, and its design pattern was replicated across other teams, significantly improving the overall quality of our inference platform."

(Feedback - For Self-Improvement)

"Looking back, while the technical outcome was perfect, the process could be improved. I could have done a better job socializing the why behind the high standard from the very beginning. Next time, I would proactively create a short document or host a brief workshop illustrating, with concrete examples, how a tiny numerical error in a low-level kernel can lead to catastrophic outcomes in fields like autonomous driving or quantitative finance. This would have built even stronger team-wide conviction earlier in the process, potentially reducing the initial resistance and aligning everyone on the 'unreasonably high' standard from the start. This is about Earning Trust through clear communication and Hiring and Developing the Best by mentoring the team on the importance of foundational quality."

#-----
insist on the highest standards, story 2
#-----

Use the example in coupang for online offline A/B test discrepancy

####-----
LP
Earn trust
####-----

EARN TRUST

Definition and Indicators

Earn Trust

Leaders listen attentively, speak candidly, and treat others respectfully. They are vocally self-critical, even when doing so is awkward or embarrassing. Leaders do not believe their or their team's body odor smells of perfume. They benchmark themselves and their teams against the best.

Concern	Mild Concern	Mixed	Mild Strength	Strength
- Fails to treat others and their ideas with respect - Blames others for mistakes - Denies or covers up mistakes - Publicly criticizes or humiliates others			- Builds positive working relationships by treating others and their ideas with respect - Seeks out and accepts feedback for self or team - Takes responsibility for shortfalls - Openly acknowledges mistakes - Provides feedback to others in a respectful manner - Represents data and information entirely transparently - Honors commitments and makes good on promises	

Earn Trust - Suggested Behavioral Interview Questions

Please choose 2-3 questions below to prepare for your interview.

- Tell me about a time when you had to communicate a change in direction that you anticipated people would have concerns with.** What did you do to understand the concerns and mitigate them? Were there any changes you made along the way after hearing these concerns? How did you handle questions and/or resistance? Were you able to get people comfortable with the change?
- Give me an example of a tough or critical piece of feedback you received.** What was it and what did you do about it?
- Describe a time when you needed to influence a peer who had a differing opinion about a shared goal.** What did you do? What was the outcome?
- Give me an example of a time when you were not able to meet a commitment.** What was the commitment and what were the obstacles that prevented success? What was the impact to your customers/peers and what did you learn from it?
- Tell me about a time when your team's goals were out of alignment with another team you relied on in order to meet your goal.** How did you work with the other team? Were you able to achieve your goals?
- Tell me about a time when you uncovered a significant problem in your team.** What was it and how did you communicate it to your manager and other stakeholders? What did you do to address the problem? How did you manage the impact of this problem for the rest of your team?
- Describe a time when you improved morale and productivity on your team.** What were the underlying problems and their causes? How did you prevent them from negatively impacting the team in the future?
- Tell me about a time when a team member was struggling to keep up or fit in and you stepped in to help out.** Why did you think they were struggling or not fitting in? Why did you decide to step in and support? What did you do to help out? How did it impact your work? What was the outcome? What did you learn from that situation?
- Tell me about a time when a team member was not performing well and impacted your work.** How did you handle that situation? Why were they not performing well? What was the outcome? What did you learn from that situation?

EARN TRUST

Definition and Indicators

Earn Trust

Leaders listen attentively, speak candidly, and treat others respectfully. They are vocally self-critical, even when doing so is awkward or embarrassing. Leaders do not believe their or their team's body odor smells of perfume. They benchmark themselves and their teams against the best.

Concern	Mild Concern	Mixed	Mild Strength	Strength
- Fails to treat others and their ideas with respect - Blames others for mistakes - Denies or covers up mistakes - Publicly criticizes or humiliates others			- Builds positive working relationships by treating others and their ideas with respect - Seeks out and accepts feedback for self or team - Takes responsibility for shortfalls - Openly acknowledges mistakes - Provides feedback to others in a respectful manner - Represents data and information entirely transparently - Honors commitments and makes good on promises	

Earn Trust - Suggested Behavioral Interview Questions

Please choose 2-3 questions below to prepare for your interview.

#-----
earn trust, story 1
#-----

Perfect  — all context locked in. Here's your **Amazon A9 “Earn Trust” STAR story** in **F3 format** (bullet + full spoken), tailored for **Senior SDE / Staff**:

Part 1: Bullet Structure Version (memorization-friendly)

Situation (1–2 min):

- While at Amazon A9, the **search relevance data pipeline** processed features for ~50 model training jobs/day.
- Partner ML teams **reported A/B test failures** due to **data delays and feature corruption**, which affected CTR metrics.
- Trust with partner teams was **low**, they questioned pipeline reliability.
- My task: **restore trust, ensure pipeline reliability, and prevent future disruptions.**

Task (1–2 min):

- Investigate root cause of data delays & corruption.
- Implement corrective action quickly to prevent further CTR impact.
- Communicate transparently with partner ML teams while resolving technical issues.
- Ensure future trust through measurable data reliability improvements.

Action (5–6 min): Options + Risk Analysis + Decision

- **Option A: Quick patch**
 - Fix individual data issues without full investigation.
 -  Pros: Fast, restores some immediate functionality
 -  Cons: Does not prevent recurrence, low partner trust
 - Risk: Another failure → trust erosion
- **Option B: Deep dive + full pipeline overhaul**  Selected
 - Analyze entire data flow, identify dependency issues, implement **data validation checkpoints**, logging, and automatic alerts
 -  Pros: Prevents recurrence, restores partner trust, aligns with **Earn Trust + Dive Deep + Highest Standards**
 -  Cons: Slower initially
 - Risk: Delay in short-term jobs, mitigated by running **backup pipeline** in parallel
- **Option C: Outsource patching to another team**
 -  Pros: Offload work
 -  Cons: Risk misalignment, slower feedback loop, weaker trust-building
- **Decision:** Option B chosen
 - **Why:** Critical to restore partner confidence, ensure long-term reliability, and demonstrate ownership.
 - **Backup plan:** Run temporary data replay pipeline; manually validate top 10% of models to prevent immediate CTR drops.

Implementation Steps:

1. Identify failing feature transformations causing corruption.
2. Add **automated validation and checksum** for each feature batch.
3. Introduce **pipeline monitoring dashboards** visible to partner ML teams.

4. Communicate daily progress and metrics to partner team until trust restored.
5. Retrospective: documented incident, created SOPs, knowledge sharing.

Result (1–2 min):

- **CTR recovery:** Top 50 model A/B tests restored → previous performance metrics returned.
- **Pipeline reliability:** Zero feature corruption incidents in 3 months post-deployment.
- **Trust:** Partner ML teams explicitly acknowledged improved transparency and confidence in pipeline.
- **Leadership recognition:** Highlighted in team-wide knowledge sharing session.
- **Customer impact:** Indirectly improved user search relevance and shopping experience.

Feedback / Reflection:

- **Learned:** Early proactive communication with partner teams could have prevented initial trust erosion.
- **Next time:** Introduce **automated alerts and validation before production rollout**, even for minor pipeline changes.

Part 2: Full Spoken Script (8–10 min)

"At Amazon A9, I owned the search relevance data pipeline that processed features for around 50 model training jobs per day. One day, partner ML teams reported A/B test failures due to delayed data and corrupted features, which negatively affected CTR metrics. Trust in my pipeline was low, and teams were questioning reliability.

My task was to quickly investigate the root cause, restore pipeline reliability, and rebuild trust with the partner teams. Given the urgency, I evaluated three options:

Option A was to quickly patch the failing data transformations. This would have restored some functionality fast, but it wouldn't prevent recurrence and could further erode trust if the issue recurred. Option C was to delegate patching to another team, which might have slowed feedback and weakened partner confidence. Option B, which I chose, was to do a deep dive and overhaul the pipeline with automated validation, checksums, and monitoring dashboards.

I implemented Option B by first identifying the exact transformations causing corruption, then adding automated validation for each feature batch, and establishing monitoring dashboards

visible to partner ML teams. I communicated progress daily to maintain transparency. As a backup plan, we ran a temporary data replay pipeline and manually validated the top 10% of models to prevent immediate CTR impact.

The results were significant. All affected A/B tests were restored to prior CTR performance, the pipeline ran reliably without feature corruption for the next three months, and partner ML teams explicitly acknowledged increased trust and transparency. Indirectly, this improved user search relevance and shopping experience.

*Looking back, I learned that **early proactive communication** with partner teams could have prevented the initial trust erosion. Next time, I would introduce **automated alerts and validation before production rollout**, even for minor changes. Overall, this experience strengthened my ability to earn trust, take ownership, and deliver results under pressure."*

If you want, I can also **create a concise 1-slide visual summary** of this story, highlighting **Situation → Action (Options + Risk) → Result → Feedback**, perfect for a **technical debrief / onsite interview prep**.

#-----
earn trust, story 2
#-----

We'll focus on **Microsoft Shopping Recommendation project**, where trust with international partner teams was critical.

Part 1: Bullet Structure Version (memorization-friendly)

Situation (1–2 min):

- While leading **shopping recommendation models** at Microsoft, partner teams in **US, Beijing, and India** reported **unexpected drops in CTR** on international markets.
- Multiple teams relied on my **recommendation model pipeline**, and inconsistent feature handling across regions caused **loss of confidence**.
- Task: **restore partner trust**, ensure model consistency, and prevent misalignment across international markets.

Task (1–2 min):

- Investigate discrepancies in feature pipelines across regions.
- Implement **processes to ensure consistent model outputs**.
- Communicate transparently with partners about causes, mitigation, and timelines.

Action (5–6 min: Options + Analysis)

Option A: Quick patch per region

- Fix regional feature pipeline individually, deploy changes.
-  Pros: Fast local improvement
-  Cons: Does not address root cause, risk of future discrepancies
- Risk: Partner trust only partially restored

Option B: Full pipeline review & alignment Chosen

- Audit all feature pipelines (text, image, L0/L1 features) across US, Beijing, India.
- Standardize preprocessing and validation, add automated **cross-region consistency checks**.
- Deploy new monitoring dashboards visible to partner teams.
-  Pros: Solves root cause, rebuilds trust, ensures long-term reliability
-  Cons: Initial slower rollout
- Risk: Minor delay in short-term deployments → mitigated by **staggered rollout**

Option C: Assign regional owners to patch pipelines independently

-  Pros: Offloads work
-  Cons: Lack of centralized visibility, slow cross-region trust recovery

Decision: Option B selected

- **Why:** Ensures trust by solving root cause, prevents recurring issues, demonstrates ownership.

- **Backup Plan:** Roll back to last stable pipeline version while new consistency checks run in parallel.

Implementation Steps:

1. Audit all regional feature pipelines.
2. Standardize preprocessing & feature engineering logic.
3. Implement automated cross-region validation & dashboards.
4. Daily syncs with partner teams for progress transparency.
5. Knowledge sharing and SOP documentation for international teams.

Result (1–2 min):

- CTR on international markets improved by **30%**.
- Partner teams regained confidence in pipeline reliability.
- Monitoring dashboards reduced **feature inconsistency incidents to zero**.
- Leadership recognized initiative for **cross-team alignment and trust-building**.

Feedback / Reflection:

- Learned: **Earlier cross-region communication** could have prevented the initial trust dip.
- Next time: Introduce **pre-deployment consistency checks** for all international pipelines to proactively earn trust.

####-----
 LP
 think big
 ####-----

THINK BIG

Definition and Indicators				
Think Big				
<i>Thinking small is a self-fulfilling prophecy. Leaders create and communicate a bold direction that inspires results. They think differently and look around corners for ways to serve customers.</i>				
Concern	Mild Concern	Mixed	Mild Strength	Strength
- Focuses on day-to-day delivery without considering long-term strategy - Fails to see opportunities and possibilities - Focuses mostly on the problems associated with a project rather than opportunities and possibilities - Does not consider the big picture or larger goals - Does not propose new projects or ideas			- Identifies bold, yet rational and defensible directions, even if the steps to get there are unclear - Takes risks when necessary in pursuit of an idea - Carefully reviews of existing processes - Able to work around limitations or challenges - Tries to think about a problem from a new perspective	

Think Big - Suggested Behavioral Interview Questions
Please choose 2-3 questions below to prepare for your interview.

- Tell me about time when you were working on an initiative or goal and saw an opportunity to do something much bigger or better than the initial focus.** Did you take that opportunity? Why or why not? What was the outcome?
- Give me an example of how you have changed the direction or view of a specific function/department and helped them embrace a new way of thinking.** Why was a change needed? What was the outcome?
- Give me an example of a time you proposed a novel approach to a problem.** What was the problem and why did it require a novel approach? Was your approach successful?
- Tell me about a time when you drove adoption for your vision/ideas.** How did you know your vision/idea was adopted by others? How did you drive adoption for your vision/ideas? How did you track adoption? Would you do anything differently?
- Give me an example of an idea or vision you had which was adopted by global stakeholders.** How did you drive adoption for your vision/ideas? How did you track adoption? Would you do anything differently?
- Tell me about a time when you thought differently to improve a process that was working.** What assumptions did you have to question? How did you evaluate if the change improved the process? Knowing what you know now, would you do anything differently?
- Tell me about a time when you encouraged a team member or organization to take a big risk.** How did you balance the risk against existing business goals? What was the outcome? What did you learn from this situation? (Manager)
- Tell me about a time when you established a vision for a (team, product, initiative) when there wasn't one.** How did you gain buy-in and drive execution? What was the outcome? Would you have done anything differently? (Manager)

THINK BIG

Definition and Indicators				
Think Big				
<i>Thinking small is a self-fulfilling prophecy. Leaders create and communicate a bold direction that inspires results. They think differently and look around corners for ways to serve customers.</i>				
Concern	Mild Concern	Mixed	Mild Strength	Strength
- Focuses on day-to-day delivery without considering long-term strategy - Fails to see opportunities and possibilities - Focuses mostly on the problems associated with a project rather than opportunities and possibilities - Does not consider the big picture or larger goals - Does not propose new projects or ideas			- Identifies bold, yet rational and defensible directions, even if the steps to get there are unclear - Takes risks when necessary in pursuit of an idea - Carefully reviews of existing processes - Able to work around limitations or challenges - Tries to think about a problem from a new perspective	

Think Big - Suggested Behavioral Interview Questions
Please choose 2-3 questions below to prepare for your interview.

```
#-----  
# think big, story 1  
#-----
```

Example 1: LLM Inference Platform Optimization (Research Institute, Hong Kong)

S – Situation:

At the research institute, we had multiple LLM inference platforms deployed for different customers. The existing system was performing sub-optimally: kernel execution was slow, memory issues occasionally caused crashes, and overall latency was above our target **TTFT/TPOT constraints**.

T – Task:

As the lead architect, my task was to **think beyond incremental improvements** and find a way to significantly improve inference performance (speed, stability, and memory efficiency) across all LLM platforms while keeping accuracy intact.

A – Action:

I explored **three main options**:

- **Option A:** Optimize existing kernels using low-level tweaks and compiler flags.
 - **Pros:** Quick to implement; low risk; maintains current platform structure.
 - **Cons:** Limited performance gains (only ~10-15% improvement).
 - **Risk:** Might not meet TTFT targets for larger LLMs.
- **Option B:** Implement **Duquant and Hadamard transpose quantization algorithms** across kernels.

- **Pros:** Potentially large performance gains (~30%), cross-platform applicability.
- **Cons:** More complex; risk of numerical instability and accuracy loss.
- **Risk:** Memory corruption due to kernel computation inaccuracies.
- **Option C:** Re-architect the LLM inference engine, using parallelism and prefetching strategies.
 - **Pros:** Could yield the **highest performance improvement**, scalable to future models.
 - **Cons:** High engineering effort; long delivery time; may disrupt current production.
 - **Risk:** Significant project delay if initial implementation fails; needs extensive regression testing.

Decision:

I chose **Option B** because it balanced **performance gains with delivery feasibility**. To mitigate risks:

1. Conducted unit tests on kernel computations to catch memory issues early.
2. Built a fallback path to Option A in case quantization caused unacceptable accuracy loss.
3. Designed a monitoring system to detect anomalies in inference latency or accuracy.

R – Results:

- Achieved **30% performance improvement** across multiple LLM inference platforms.
- Maintained accuracy within acceptable bounds.
- Provided a reusable kernel optimization framework for future models.

Feedback:

Next time, I could **combine Option B + C incrementally**, implementing quantization first and then parallel prefetching in a phased approach to further push performance gains while reducing delivery risk.

LP Mapping: Think Big, Deliver Results, Dive Deep, Bias for Action.

```
#-----
# think big, story 2
#-----
```

Example 2: Multi-market Recommendation System (Microsoft, Shopping Ads)

S – Situation:

The international shopping recommendation system under my leadership had **subpar CTR in some regions**, and existing models were struggling to combine text and image features efficiently.

T – Task:

I needed to **think big** and design a **next-generation recommendation agent** capable of significantly increasing CTR/DAU and scaling across markets.

A – Action:

I evaluated three options for the recommendation architecture:

- **Option A:** Tune existing DeepFM/DIN models with more features.
 - **Pros:** Low-risk, incremental improvement.
 - **Cons:** Limited performance; would not fully leverage multimodal data.
- **Option B:** Implement **multi-modal L1/L2 models** combining text and image features, using generative AI for personalization.
 - **Pros:** Significant CTR potential; innovative; sets foundation for future personalization.
 - **Cons:** Complex feature engineering; longer training time; more infra required.
 - **Risk:** Overfitting in low-data markets; model serving complexity.
- **Option C:** Build **personal shopping assistant agent** using Chain-of-Thought and Tree-of-Thought combined with RAG.
 - **Pros:** Highly innovative; could create a market-leading personalized shopping experience.
 - **Cons:** Very high complexity; needs strong infrastructure and AI ops support.
 - **Risk:** Delivery time could exceed quarterly goals; user adoption risk.

Decision:

I chose **Option B** as a strategic balance of **innovation and feasibility**, while planning Option C as a longer-term roadmap. Backup plans included:

1. Fall back to Option A for markets where multimodal features underperformed.
2. Implement progressive rollout and A/B testing to mitigate user impact.

R – Results:

- CTR improved **10% on en-US, 30% on international markets**.
- Delivered a scalable framework for multimodal recommendation models.
- Laid groundwork for personal shopping agent, now in pilot testing.

Feedback:

Next time, I would **prototype Option C earlier** with a smaller subset of users to test feasibility while executing Option B at scale.

LP Mapping: Think Big, Invent & Simplify, Deliver Results, Customer Obsession.

####-----
LP
have backbone, disagree and commit
####-----

HAVE BACKBONE; DISAGREE AND COMMIT

Definition and Indicators

Have Backbone; Disagree and Commit

Leaders are obligated to respectfully challenge decisions when they disagree, even when doing so is uncomfortable or exhausting. Leaders have conviction and are tenacious. They do not compromise for the sake of social cohesion. Once a decision is determined, they commit wholly.

Concern	Mild Concern	Mixed	Mild Strength	Strength
- Refuses to change direction even when data shows this is necessary - Unwilling to challenge other's decisions when necessary - Comes off as consistently argumentative - Continues to advocate original point of view even after final decision has been made - Focuses on getting their own way rather than what is best for the customer			- Disagrees with peers and supervisors using data to thoughtfully explain counter-arguments - Stands up for what they believe is in the best interest of the company and the customer - Advocates clearly and passionately for beliefs - Openly supports and commits to decisions that have been made, even if not originally in agreement	

led you to commit to the decision? Would you make the same decision now?

- Describe a time when you felt really strongly about something on a project but the team decided to go in a different direction. How hard did you press the issue? How did you approach that project afterward?
- Tell me about a time when you pushed back against a decision that negatively impacted your team. What was the issue? How did it turn out? Would you have done anything differently?
- Give me an example of when you submitted a great idea to your manager and they did not support it. What was the idea? How did you handle the lack of support?
- Describe a time when you had to support a business initiative that you didn't agree with. How did you handle it? How did you deliver the message to your team?
- Tell me about a time when the business gained something because you persisted for a length of time. Why were you so determined? How did it turn out?

#-----
 # have backbone, disagree and commit, story 1
 #-----

Example 1: LLM Kernel Quantization Decision (Research Institute, Hong Kong)

S – Situation (1–2 min):

While improving our LLM inference platform, I found that the proposed approach by the team lead was to incrementally optimize existing kernels with compiler-level tweaks. The expected gains were small (~10%), and the team was satisfied with just meeting minimum latency targets. I believed we could achieve **much larger performance gains (~30%)** by implementing **Duquant and Hadamard transpose quantization algorithms**, but this approach had risks of memory corruption and numerical instability.

T – Task (1 min):

I needed to **convince leadership to adopt a riskier but higher-reward approach** while maintaining delivery timelines and avoiding production issues.

A – Action (5–6 min):

I analyzed three options:

- **Option A – Incremental optimization (team lead’s choice):**
 - Pros: Low risk, quick delivery, minimal disruption.
 - Cons: Limited performance gain; wouldn’t meet long-term scaling needs.
 - Risk: Could underdeliver on performance metrics for new models.
- **Option B – Kernel quantization (my proposed choice):**
 - Pros: High performance gain (~30%), reusable across platforms.
 - Cons: Medium complexity; risk of memory corruption.
 - Risk: Could break inference for large LLMs; debugging could delay delivery.
- **Option C – Full re-architecture with parallelism and prefetching:**
 - Pros: Potentially highest performance.
 - Cons: Very high effort; long timeline; could disrupt production.
 - Risk: Delivery risk; high regression test cost.

Decision:

I **disagreed with the team lead and advocated for Option B**, presenting a **risk mitigation plan**:

1. Conduct unit testing for numerical stability.
2. Implement fallback path to Option A if accuracy dropped below threshold.
3. Monitor production metrics with alerting to detect anomalies.

After discussions with stakeholders, I **had backbone** and convinced them to adopt Option B while **committing fully** once the decision was made.

R – Results (1–2 min):

- Achieved **30% performance improvement** across multiple LLM platforms.

- Accuracy maintained within acceptable bounds.
- Team gained a framework reusable for future models.

Feedback:

Next time, I would prepare a **smaller-scale prototype earlier** to demonstrate feasibility before the full rollout, which could have shortened the decision cycle.

LP Mapping: Have Backbone; Disagree & Commit, Think Big, Deliver Results, Dive Deep.

#-----
 # have backbone, disagree and commit, story 2
 #-----

Example 2: Multi-Market Recommendation Feature Inclusion (Microsoft, Shopping Ads)

S – Situation (1–2 min):

During a cross-market project, the team proposed **excluding image features** in L2 recommendation models for certain low-traffic markets, citing speed concerns. I believed that **including multi-modal features (text + image)** would significantly improve CTR and DAU even in smaller markets, but this approach required extra computation and infrastructure.

T – Task (1 min):

I had to **advocate for multi-modal features**, balancing performance improvements with infrastructure constraints and delivery schedule.

A – Action (5–6 min):

Options considered:

- **Option A – Follow original plan (text-only):**
 - Pros: Faster to implement, minimal infrastructure change.
 - Cons: Limited CTR improvement; inconsistent user experience across markets.
- **Option B – Include multi-modal features (my proposal):**

- Pros: Higher CTR/DAU potential; consistent experience.
- Cons: Medium complexity; extra cost in model serving and storage.
- Risk: Overfitting in low-data markets; serving latency.
- **Option C – Incremental rollout (hybrid):**
 - Pros: Safer; test impact in selected markets.
 - Cons: Slower overall improvement; might delay global impact.

Decision:

I **disagreed with the initial plan** and proposed **Option B with phased rollout as mitigation**, committing fully once leadership approved. Backup plan: roll back to Option A for markets where latency exceeded thresholds.

R – Results (1–2 min):

- CTR improved **10% in US, 30% internationally**.
- System scaled well; prepared foundation for future personal shopping assistant agent.

Feedback:

Next time, I would **prepare simulation data comparing Option A and B impacts** in advance to make the disagreement stronger and reduce decision debate time.

LP Mapping: Have Backbone; Disagree & Commit, Customer Obsession, Deliver Results, Invent & Simplify.

####-----
 LP
 ownership
 ####-----

OWNERSHIP

Definition and Indicators

Ownership

Leaders are owners. They think long term and don't sacrifice long-term value for short-term results. They act on behalf of the entire company, beyond just their team. They never say "that's not my job."

Concern	Mild Concern	Mixed	Mild Strength	Strength
- Makes decisions for short-term team success rather than long-term value - Describes setbacks or problems without talking about how to address them - Relies on others to remove roadblocks - Avoids tough decisions - Avoids addressing problems that are in others' work areas - Blames others for setbacks - Assigns work to others because the work is undesirable - Walks away if there are too many difficulties - Creates solutions that negatively impact other teams			- Actively makes improvements outside of one's area of responsibility - Makes decisions that consider risks and future outcomes - Makes decisions that are scalable and contribute to long-term success - Takes the lead in solving problems - Takes accountability for dependencies and their work - Takes ownership for mistakes - Accepts responsibility - Inspires others to take ownership - Sees things through to completion	

Ownership - Suggested Behavioral Interview Questions

Please choose 2-3 questions below to prepare for your interview.

1. Tell me about a time when you took on something significant outside your area of responsibility. Why was it important? What was the outcome?

2. Describe a time when you didn't think you were going to meet a commitment you promised. How did you identify the risk and communicate it to stakeholders? Is there anything you would do differently?

3. Give me an example of an initiative you undertook because you saw that it could benefit the whole company or your customers, but wasn't within any group's individual responsibility so nothing was being done.

4. Tell me about a time when you made a hard decision to sacrifice short term gain for something that would create long term value for the business. What was the outcome? Knowing what you know now, would you have done anything differently?

5. Describe a time when you had to transition a project you owned to a new owner. What steps did you take to make sure the transition went smoothly? Tell me about a time that you chose to get involved in a project that you had already transitioned to somebody else. What was the situation? Why was it important to get involved?

6. Tell me about a time when you saw a peer struggling and decided to step in and help. What was the situation? Why did you decide to step in? What actions did you take?

#-----
ownership, story 1

#-----

Example 1: End-to-End Ownership of Search Pipeline (Amazon-A9, Palo Alto)

S – Situation (1–2 min):
At Amazon-A9, I owned a **data pipeline whose output was consumed by 95% of all search relevance models**. I noticed that inconsistencies in upstream data were causing **search model performance fluctuations**, impacting millions in profit. The team treating this as a minor issue was focusing only on individual model fixes, not the pipeline.

OWNERSHIP

Definition and Indicators

Ownership

Leaders are owners. They think long term and don't sacrifice long-term value for short-term results. They act on behalf of the entire company, beyond just their team. They never say "that's not my job."

Concern	Mild Concern	Mixed	Mild Strength	Strength
- Makes decisions for short-term team success rather than long-term value - Describes setbacks or problems without talking about how to address them - Relies on others to remove roadblocks - Avoids tough decisions - Avoids addressing problems that are in others' work areas - Blames others for setbacks - Assigns work to others because the work is undesirable - Walks away if there are too many difficulties - Creates solutions that negatively impact other teams			- Actively makes improvements outside of one's area of responsibility - Makes decisions that consider risks and future outcomes - Makes decisions that are scalable and contribute to long-term success - Takes the lead in solving problems - Takes accountability for dependencies and their work - Takes ownership for mistakes - Accepts responsibility - Inspires others to take ownership - Sees things through to completion	

Ownership - Suggested Behavioral Interview Questions

Please choose 2-3 questions below to prepare for your interview.

T – Task (1 min):

I decided to **take full ownership of the pipeline** to fix root causes, improve stability, and reduce manual intervention, ensuring consistent model performance across all downstream models.

A – Action (5–6 min):

I considered three approaches:

- **Option A – Quick patch per model:**
 - Pros: Immediate relief; minimal team disruption.
 - Cons: Repetitive, does not fix root cause; manual overhead remains.
 - Risk: Could fail again when data sources change.
- **Option B – Full pipeline revamp (my choice):**
 - Pros: Solves root cause; reduces manual workloads; scalable.
 - Cons: More development effort; needs cross-team coordination.
 - Risk: Potential temporary disruption during migration; requires careful testing.
- **Option C – Incremental fixes with monitoring:**

- Pros: Medium effort; gradually improves system.
- Cons: Slower results; may take months to fully stabilize.
- Risk: Business impact continues until full stabilization.

Decision:

I chose **Option B: revamp the pipeline**, implementing robust ETL checks, standardized data formats, and automated alerts. Backup plan: if migration caused major disruptions, we could temporarily revert to the old pipeline while fixing issues.

R – Results (1–2 min):

- Reduced **manual model retraining workloads by 20%**.
- Reduced **model training time by 20%**.
- Prevented profit loss due to inconsistent search relevance models.
- Enabled downstream teams to rely on stable data and focus on model innovation.

Feedback:

Next time, I would **engage stakeholders earlier** from other teams during the planning phase to identify hidden dependencies and accelerate buy-in.

LP Mapping: Ownership, Deliver Results, Dive Deep, Invent & Simplify, Bias for Action.

#-----
 # ownership, story 2
 #-----

Example 2: Shopping Recommendation Model Deployment (Microsoft, Shopping Ads)

S – Situation (1–2 min):

While leading the international shopping recommendation project, our **CTR in certain regions was underperforming**. Other teams suggested **delaying rollout** until all models were perfectly tuned. I saw an opportunity to **take ownership of the problem end-to-end**, from model design to online deployment and A/B testing.

T – Task (1 min):

Ensure that **model improvements reach users quickly**, while continuously monitoring performance and iteratively optimizing for each market.

A – Action (5–6 min):

Options considered:

- **Option A – Wait for perfect model before rollout:**
 - Pros: Low risk; avoids initial failures.
 - Cons: Delays value to users; lost CTR opportunities.
- **Option B – Incremental rollout with monitoring (my choice):**
 - Pros: Allows faster delivery; learn and adapt from real user feedback.
 - Cons: Requires robust monitoring; slight risk of negative CTR impact initially.
 - Risk: Could temporarily reduce CTR in some markets.
- **Option C – Delegate to regional teams:**
 - Pros: Local expertise; reduces central workload.
 - Cons: Inconsistent approach; slower global learning; harder to maintain standards.

Decision:

I chose **Option B**, taking full ownership of rollout and monitoring. I implemented **automated dashboards and alerts** to detect issues early and quickly apply fixes. Backup plan: rollback underperforming models in any region without affecting global deployment.

R – Results (1–2 min):

- CTR improved **10% in US, 30% in international markets**.
- System scaled effectively; teams could trust model predictions.
- Set foundation for **personal shopping assistant agent** using CoT/ToT and RAG.

Feedback:

Next time, I would **document rollout decisions and risk mitigation in advance**, to accelerate cross-team approvals and reduce debate time.

LP Mapping: Ownership, Customer Obsession, Deliver Results, Dive Deep, Bias for Action.

####-----
 LP
 Customer obsession
 ####-----

CUSTOMER OBSESSION

Definition and Indicators

Customer Obsession				
<i>Leaders start with the customer and work backwards. They work vigorously to earn and keep customer trust. Although leaders pay attention to competitors, they obsess over customers.</i>				
Concern	Mild Concern	Mixed	Mild Strength	Strength
• Makes decisions or takes actions without adequately considering the customer impact • Fails to collect or respond to customer input appropriately • Fails to meet customer commitments • Loses and doesn't regain customers' trust • Develops a project approach based on industry trends rather than customer needs • Unable to identify primary customers			• Takes actions guided by customer input • Works backwards from the optimal customer experience • Tries to surprise and delight the customer • Applies time and energy to make the customer experience more efficient or enjoyable • Identifies new ways of gathering feedback from customers • Listens to feedback from customers and uses it to make improvements • Pushes back when necessary to ensure that decisions consider what is best for customer in the long-term • Earns customers trust by delivering promised services and products which meet or exceed expectations • Stops activities that no longer enhance the customer experience	

Customer Obsession - Suggested Behavioral Interview Questions
 Please choose 2-3 questions below to prepare for your interview.

1. **Describe a difficult interaction you had with a customer.** How did you deal with it? What was the outcome? How would you handle it differently?
2. **Tell me about a time when you went above and beyond for a customer.** Why did you do it? How did the customer respond? What was the outcome?
3. **Give me an example of when you were able to anticipate a customer need with a solution/product they didn't know they needed/wanted yet.** How did you know they needed this? How did they respond?
4. **Give me an example of a time when you asked for customer feedback.** How did you use that feedback to drive innovation or improvement? How did the customer respond?
5. **Tell me about a time when you evaluated the customer experience of your product or service.** What did you do? What was the result?
6. **Tell me about a time when a customer came to you for something that wouldn't actually address their need.** How did you approach the situation? What was the result?
7. **Sometimes customers make unreasonable requests. Tell me about a time when you've had to push back or say no to a customer request.** What did you say or do in response to that request?
8. **Tell me about a time when you had to balance the needs of the customer with the needs of the business.** What did you do? What was the result?
9. **Give me an example of a change you implemented in your current team or organization to meet the needs of your customers.** What has been the result?
 (Manager)

CUSTOMER OBSESSION

Definition and Indicators

Customer Obsession				
<i>Leaders start with the customer and work backwards. They work vigorously to earn and keep customer trust. Although leaders pay attention to competitors, they obsess over customers.</i>				
Concern	Mild Concern	Mixed	Mild Strength	Strength
• Makes decisions or takes actions without adequately considering the customer impact • Fails to collect or respond to customer input appropriately • Fails to meet customer commitments • Loses and doesn't regain customers' trust • Develops a project approach based on industry trends rather than customer needs • Unable to identify primary customers			• Takes actions guided by customer input • Works backwards from the optimal customer experience • Tries to surprise and delight the customer • Applies time and energy to make the customer experience more efficient or enjoyable • Identifies new ways of gathering feedback from customers • Listens to feedback from customers and uses it to make improvements • Pushes back when necessary to ensure that decisions consider what is best for customer in the long-term • Earns customers trust by delivering promised services and products which meet or exceed expectations • Stops activities that no longer enhance the customer experience	

Customer Obsession - Suggested Behavioral Interview Questions

请根据我的简历，使用

1. STAR + feedback, feedback means if I do it next time, how could I do better?
2. 14 amazon rules来举例回答,
3. Situation and Task 1-2 minutes, Action 5-6 minutes, Results 1-2 minutes
4. for action, if possible, please list option A, B, C, and why you choose one option, why you make this decision
5. Compare option A, B, C, and their trade off, Include risk analysis
6. also include backup plan

#-----
 # customer obsession, story 1
 #-----

1. Describe a difficult interaction you had with a customer.
2. Tell me about a time when you went above and beyond for a customer .

3. Give me an example of when you were able to anticipate a customer need with a solution/ product they didn't know they needed/wanted yet .
4. Give me an example of a time when you asked for customer feedback .
5. Tell me about a time when you evaluated the customer experience of your product or service .
6. Tell me about a time when a customer came to you for something that wouldn't actually address their need.
7. Sometimes customers make unreasonable requests. Tell me about a time when you've had to push back or say no to a customer request.
8. Tell me about a time when you had to balance the needs of the customer with the needs of the business.
9. Give me an example of a change you implemented in your current team or organization to meet the needs of your customers.

#-----

Here are the 3 I propose:

Story #	Company	Topic	Angle of Customer
Obsession			
-----	-----	-----	-----

✓ Story 1	Hong Kong AI Institute	LLM Inference latency optimization	Deep technical obsession for latency + reliability for enterprise users
✓ Story 2	Microsoft Shopping Ads	Improving CTR for low-resource international markets	Obsess over underserved customers globally
✓ Story 3	Amazon A9 Search	Stability of data pipeline used by 95% ranking models	Internal customer obsession + reliability mindset

#-----
customer obsession, story 1
#-----

Situation (1–2 min)

In Microsoft Bing shopping reco team, there are always complain about the quality of recommendation. The daily unsatisfactory ratio is 16% for example, 1) kids with guns, 2) adult products to kids, 3) ...

Task
significantly reduce the unsatisfactory by using different technologies

Action (5–6 min)

Step 1 — Deep Dive into Customer Pain

get data points, collected all kind of incorrect reco

Step 2 — analyze from where this bad reco comes from

step 3 - design several solutions to filter them out

1) index layer

2) retrieval layer

3) ranking layer

step 4 - using image models to filter out bad image

Result (1–2 min)

insatisfactory ratio reduced from 16% to 1.6%, 90% reduction

internal, external complaining ratio reduced significantly

— — — — —

customer obsession, story 2

— — — — —

B — Structured Outline (Practice crib / bullets)

Title: Customer Obsession — LLM Inference Latency Optimization (Hong Kong Research Institute)

Role: Tech Lead / Software Architect (T2)

Metric Target: P95 TTFT from 180ms → 120ms

Customers impacted: 9 enterprise customers

Situation (1–2 min)

Enterprise customers reported P95 TTFT SLA violations → business risk.

Root cause: head-of-line blocking caused by >30 interdependent deployment parameters (batching, prefill thresholds, KV sizing, scheduler heuristics).

Business need: fix quickly and reliably without extra hardware.

(LPs: Customer Obsession, Earn Trust)

Task (30–60s)

Reduce P95 TTFT to <150ms (target ~120ms).

Make deployment reproducible, reduce configuration brittleness.

Timeline: 6–8 weeks.

(LPs: Ownership, Deliver Results)

Action — Options considered (5–6 min)

Option A — Brute-force / grid search

Pros: Fast to start, familiar.

Cons: Explosion of search space (30+ params), costs, brittle, reactive.

Why rejected: Doesn't generalize; keeps us tuning indefinitely. (LP: Bias for Action but low Invent & Simplify)

Option B — Model-driven tuning (Bayesian Optimization + fingerprinting)

Pros: Sample-efficient; adaptive.

Cons: Adds subsystem complexity; reactive; operational burden.

Why rejected: Better than A but still reactive and added maintenance cost.

Option C — Architectural change: prefill-aware scheduler + adaptive deterministic KV rules (Chosen)

Pros: Attacks root cause, reduces parameter interactions, robust across workloads, simpler ops.

Cons: More upfront engineering and validation.

Why chosen: Durable fix aligned with customer need and platform maintainability. (LPs: Customer Obsession, Invent and Simplify, Dive Deep, Insist on Highest Standards)

Implementation steps (led by me)

Instrumentation and decomposition of latency stages. (Dive Deep)

Design & prototype two-queue prefill-aware scheduler. (Invent & Simplify)

Sensitivity analysis → collapse 30+ params to 4 high-leverage knobs. (Insist on Highest Standards)

Automated validation suite + canary rollout + rollback path. (Bias for Action, Ownership)

Customer-facing dashboard + runbook; stakeholder coordination and PM prioritization. (Earn Trust, Deliver Results)

Result (1–2 min)

P95 TTFT: 180ms → 120ms (met and exceeded target).

9 enterprise customers' SLAs restored; no contract terminations; two customers expanded usage.

Tunable surface reduction: 30+ → 4 knobs; on-call incidents dropped ~40% next quarter.

Platform had better operator experience and customer transparency (dashboard).

LP summary: Customer Obsession (primary), Dive Deep, Ownership, Invent & Simplify, Deliver Results, Earn Trust, Insist on Highest Standards, Bias for Action.

####-----
LP
invent and simplify
####-----

#-----
invent and simplify, story 1
#-----

INVENT AND SIMPLIFY

Definition and Indicators				
Invent and Simplify				
<i>Leaders expect and require innovation and invention from their teams and always find ways to simplify. They are externally aware, look for new ideas from everywhere, and are not limited to "not invented here." As we do new things, we accept that we may be misunderstood for long periods of time.</i>				
Concern	Mild Concern	Mixed	Mild Strength	Strength
- Accepts current processes even if there is room for improvement - Creates a cumbersome process to solve a short-term problem - Allows complex processes to exist within their control - Doesn't share discoveries and insights with others - Only focuses on ideas and resources that can be gathered internally - Discourages others from exploring new ideas - Looks for a standard process or series of steps to copy - Speaks at length about the difficulty of doing something without naming ideas for a better process			- Generates new ideas or methods for solving a problem - Identifies areas for improvement and deliver new features that are scalable and avoid unnecessary complexity - Actively gathers ideas from a wide range of internal and external sources - Focuses on solutions that will impact the customer experience long-term. - Seeks out opportunities to improve a product or process.	

Invent and Simplify - Suggested Behavioral Interview Questions

Please choose 2-3 questions below to prepare for your interview.

trying to solve? How did you measure success? What was the end result/impact? (Manager)

Situation (1–2 min)

In Microsoft Bing shopping reco team, there are 2 markets: 1) us market, 2) international markets, which has > 20 countries
each country has its own laws, customs, and rules
in tech, each country has its own data pipeline, training process, etc

Task

simplify and re-organize the entire reco system, make 1) improve the the quality for existing countries, 2) making the launching process of a new country easy, 3) making the maintenance process easy

Action (5–6 min)

Step 1 — Deep Dive into process in each country
get data points, collected the pain point for existing system

Step 2 — redesign the reco system for international markets

1) maximamly combine the common part, e.g., data collection, common feature engineering

2) for each country or region, set specific process, e.g., filter sensitive data

step 3 - small step, fast iteration. first include the first country in the new system, do it end to end, make sure the process is working. then include more countried one by one

step 4 - has backup plan. if something fails, quickly cut the country back to the original system

Result (1–2 min)

1. the time to launch a new country reduced from 2 weeks to 2-3 days
2. make the system maintenance much easier and simpler, normally any working item takes 1-2 weeks, now 1-2days
3. much easier to launch new ML models in the new international markets, significantly improve user experience. after using the new system, I improved ctr and dau in international markets 10%

#-----
invent and simplify, story 2
#-----

Example: Simplifying the Global Recommendation Model Architecture

Amazon Leadership Principle in Focus: Invent and Simplify

Leaders expect and require innovation and invention from their teams and always find ways to simplify. They are externally aware, look for new ideas from everywhere, and are not limited by “not invented here.” As we do new things, we accept that we may be misunderstood for long periods of time.

(Situation & Task - 1.5 minutes)

"In my role as Principal Applied Scientist at Microsoft, leading the shopping recommendation and ads projects, we faced a significant scaling challenge. Our strategy for international expansion had been to build and maintain increasingly complex, region-specific models. We had separate DeepFM and DIN models for the US, India, Brazil, and others. While this initially boosted performance, it created a massive operational burden. Each model had its own training pipeline, feature set, and deployment cycle. Launching a global feature update required coordinating across 5+ different codebases and model pipelines, which was slow, error-prone, and resource-intensive. The system was becoming a complex 'hairball' that was hindering our velocity.

The task was clear: we needed to reinvent our recommendation architecture to enable rapid, global innovation while maintaining or improving performance. We needed a solution that was both powerful in its capabilities and radically simpler in its operation."

(Action - 5-6 minutes)

"I led a team to brainstorm architectures that could unify our global efforts. We evaluated three main options:

Option A: The Meta-Ensemble. Keep all existing regional models but build a new global "meta-model" on top that would learn to weight the predictions of the regional experts. This is a classic ensemble approach.

Option B: The Monolithic Global Model. Consolidate all data into one massive, global training set and train a single, gigantic model to serve all traffic, using region as just one more feature.

Option C: Invent a Simplified Multi-Task Architecture. Design a new model with a shared foundational backbone for learning universal user-item concepts, and lightweight, region-specific "adapter" heads for local customization. This is a "backbone + head" paradigm inspired by transfer learning.

Comparison and Trade-off Analysis:

Option A (Meta-Ensemble):

Pros: Leveraged all our existing investments. Non-destructive and low immediate risk.

Cons & Risks: This was the opposite of simplification. It added a new, complex layer (the meta-model) on top of the existing complexity. It doubled our inference cost and latency. It was a patch, not a solution, and would make the system even harder to debug and maintain.

Why I didn't choose it: It failed the "simplify" test entirely. It was an incremental complication, not a fundamental re-invention.

Option B (Monolithic Model):

Pros: Conceptually simple: one model to rule them all. Eliminates the overhead of managing multiple models.

Cons & Risks: Extremely high risk of model "amnesia" or negative transfer, where strong signals from the US market could drown out subtler patterns in smaller regions. Training would be incredibly computationally expensive. Debugging why a recommendation was made in a specific region would be very difficult. It was a brute-force approach that ignored the nuanced needs of our customers.

Why I didn't choose it: While it simplified the number of models, it created a monolithic, black-box system that was unlikely to perform well for all customers. It wasn't truly customer-obsessed.

Option C (Multi-Task Architecture):

Pros: This was the only option that truly embodied Invent and Simplify. We were inventing a new, modular architecture for global ML systems. It simplified our MLOps dramatically: we now had one core model to update for global features and lightweight, low-risk adapters for regional tests. It was externally aware, drawing inspiration from computer vision and NLP's use of adapters and multi-task learning.

Cons & Risks: This was a novel R&D effort with no guarantee of success. We had to design the interface between the backbone and the adapters and ensure the backbone learned generalized, not US-biased, representations.

Why I chose this: It was a principled bet on a superior long-term architecture. It simplified our engineering processes while inventing a more elegant and powerful technical solution. It allowed us to "be misunderstood" by those who favored the proven-but-complex approach of separate models, as we built the case for this new paradigm.

My Decision and Execution:

I championed Option C. I designed the architecture where the shared backbone was a transformer-based sequence model that processed user history to learn universal engagement patterns. The region-specific heads were small, feed-forward "adapter" networks that took the backbone's embeddings and specialized them for local context. We applied Multi-Modality by having the backbone also generate fused text/image embeddings that the adapters could use.

Backup Plan: I was aware of the performance risk. Our backup plan was to run the new system in a "shadow mode" for one month, where it generated recommendations in parallel with the production models but did not serve them to users. We would compare its outputs against the live models. If the new architecture failed to meet performance thresholds, we would pivot to a hybrid approach: use the new unified backbone but fall back to the older, proven regional heads for the most critical markets, giving us more time to refine the adapter design."

(Results - 1-2 minutes)

"The reinvention was a resounding success. The new multi-task architecture reduced the operational overhead of launching a global feature by over 60%, as we now only had to update the core backbone model. This dramatically increased our team's iteration velocity.

In terms of performance, the system not only maintained but improved overall recommendation quality. We saw an additional 5% CTR gain in our international markets because the shared backbone, trained on global data, actually learned richer, more robust representations that benefited the smaller regions. We had successfully simplified a complex operational nightmare into an elegant, scalable, and more performant system. This architecture later became the foundation for incorporating Generative AI and conversational agents into the recommendation flow."

(Feedback - For Self-Improvement)

####-----
LP
Deliver results
####-----

DELIVER RESULTS

Definition and Indicators

Deliver Results				
<i>Leaders focus on the key inputs for their business and deliver them with the right quality and in a timely fashion. Despite setbacks, they rise to the occasion and never settle.</i>				
Concern	Mild Concern	Mixed	Mild Strength	Strength
- Fails to deliver on required commitments - Does not commit adequate time and resources to deliver quality results - Settles for work products that do not meet expectations - Makes excuses for why things cannot be done before a deadline - Delivers projects late or with missing requirements			- Meets deadlines and expectations with products that are well-executed and high-quality - Focuses on delivering the most important products - Able to deliver multiple projects at the same time - Overcomes and persists in the face of obstacles - Communicates regularly about the status of projects	

Deliver Results - Suggested Behavioral Interview Questions

Please choose 2-3 questions below to prepare for your interview.

- Tell me about a time when you did not effectively manage your projects and something did not get completed on time. What was the impact? What approaches do you use to make sure you are focusing on the right deliverables when you have several competing priorities?
- It can be difficult to set goals for a team that are challenging, yet achievable. Tell me about a time when you hit the right balance. How did you approach setting the goals? What was the outcome? (Manager)

—————
 # Deliver results, story 1
 # —————

Leading product search Improvements in coupang (Principal machine learning engineer)

Situation (1-2 minutes):

- in early 2019, coupang almost bankrupt
- our ceo bom asked us to use all methods to improve revenue
- it is emergency situation

Task (1-2 minutes):

- train new ML model to improve revenue

Action (5-6 minutes):

- As Principal Applied Scientist, I did dive deep investigation of our system.
- I evaluated several approaches, finally decide to add more personalized features.
 e.g., in gracery category, people like to but cheap products, so return cheap products to them.
 for fashion category, people like new in season style, adding more fashion products when customer search for fashion products, etc. similar as Amazon categorical search

3. quickly trained a new model, adding more personalized feature, in 3 weeks.

results:

1. the first model got 2.5M \$ per quarter, 10M \$ per year, saved company
2. I got a prize from our svp.
3. after this model, I led both from us side and south korean side engineers, over 10 engineers, keep on improve our search system, consistently improve revenue

#-----
Deliver results, story 2
#-----

Example 1: CTR and DAU Improvement for International Shopping Recommendation (Microsoft, Mountain View)

S – Situation (1–2 min):

Our shopping recommendation system was performing well in the US market but **underperforming in international markets**, with lower CTR and engagement. Leadership was concerned about the slow rollout of model updates due to multi-region infrastructure complexity.

T – Task (1 min):

I was responsible for **driving measurable improvements in CTR and DAU internationally**, ensuring that model updates were timely and effective while minimizing deployment risk.

A – Action (5–6 min):

Options considered:

- **Option A – Incremental model update only for US market:**
 - Pros: Low risk; fast deployment.
 - Cons: No international impact; doesn't meet leadership goal.
 - Risk: Missed revenue opportunity.
- **Option B – Multi-modal recommendation model (text + image) rolled out internationally (my choice):**
 - Pros: Directly improves relevance and engagement globally; measurable CTR and DAU gains.

- Cons: Higher infrastructure complexity; possible latency increase.
- Risk: Latency or model instability affecting users.
- **Option C – Aggressive rollout of personal shopping assistant agent (CoT + ToT + RAG):**
 - Pros: Potentially large impact; innovative solution.
 - Cons: High complexity; long delivery timeline; high risk.
 - Risk: Delayed results; deployment failures.

Decision:

Chose **Option B** with phased rollout and monitoring. Backup plan: fallback to US-only deployment if latency thresholds were exceeded.

R – Results (1–2 min):

- **10% CTR increase in US, 30% in international markets.**
- DAU growth and positive user engagement metrics.
- Infrastructure and model pipeline prepared for future agent rollout.

Feedback:

Next time, I would **conduct early stress testing in international regions** to proactively prevent latency issues and ensure smoother deployment.

LP Mapping: Deliver Results, Customer Obsession, Ownership, Bias for Action, Think Big.

####-----
 LP
 Dive deep
 ####-----

#-----

DIVE DEEP

Definition and Indicators

Dive Deep

Leaders operate at all levels, stay connected to the details, audit frequently, and are skeptical when metrics and anecdotes differ. No task is beneath them.

Concern	Mild Concern	Mixed	Mild Strength	Strength
- Does not understand the details of projects - Does not understand how different groups or systems work together - Only holds surface-level understanding of metrics and data - Does not question assumptions - Unable to step in and get work done			- Stays connected to the details of projects and programs - Understands how different groups or systems work together - Critically evaluates metrics and data - Asks good questions that provide clarity to situations - Steps in and gets work done - Investigates and get details in order to solve a problem - Gathers information to solve a problem, even if it's difficult or time-consuming	

Dive Deep - Suggested Behavioral Interview Questions

Please choose 2-3 questions below to prepare for your interview.

- Tell me about a time when you were trying to understand a complex problem on your team and you had to dig into the details to figure it out** Who did you talk with or where did you have to look to find the most valuable information? How did you use that information to help solve the problem?
- Tell me about a situation that required you to dig deep to get to the root cause.** How did you know you were focusing on the right things? What was the outcome? Would you have done anything differently?
- Tell me about a problem you had to solve that required in-depth thought and analysis.** How did you know you were focusing on the right things? What was the outcome? Would you have done anything differently?
- Walk me through a big problem or issue in your organization that you helped to solve.** How did you become aware of it? What information did you gather? What information was missing and how did you fill the gaps? Did you do a reflection at the conclusion of the project? If so, what did you learn?
- Tell me about a specific metric you have used to identify a need for a change in your department.** Did you create the metric or was it already available? How did this and other information influence the change? What was the outcome of this change?
- Have you ever created a metric that helped identify a need for a change in your department?** What was the metric? Why did you create it? How did this and other information influence change? What was the outcome of the change?
- Tell me about a time when you had to validate the assumptions underlying a direct report's project.** How did you decide follow up was necessary? What steps, if any, did you take to validate the assumptions? What was the result? (Manager)

Dive deep, story 1

—————

DIVE DEEP

Definition and Indicators

Dive Deep

Leaders operate at all levels, stay connected to the details, audit frequently, and are skeptical when metrics and anecdotes differ. No task is beneath them.

Concern	Mild Concern	Mixed	Mild Strength	Strength
- Does not understand the details of projects - Does not understand how different groups or systems work together - Only holds surface-level understanding of metrics and data - Does not question assumptions - Unable to step in and get work done			- Stays connected to the details of projects and programs - Understands how different groups or systems work together - Critically evaluates metrics and data - Asks good questions that provide clarity to situations - Steps in and gets work done - Investigates and get details in order to solve a problem - Gathers information to solve a problem, even if it's difficult or time-consuming	

Dive Deep - Suggested Behavioral Interview Questions

Please choose 2-3 questions below to prepare for your interview.

#-----
Dive deep, story 1
#-----

Leading Microsoft Bing repo engine performance optimization

Situation (1-2 minutes):

1. Every time we launch a new ML model, it introduces significant end to end latency to our customer,
2. also significant CPU/GPU utilization up, triggering warning
3. This situation esp. bad when model size becoming bigger and bigger

Task (1-2 minutes):

1. root cause
2. quick fix
3. how to systematically solve this problem

Action (5-6 minutes):

1. collect lots of profiling data, across several layers, e.g., service layer, from ML model, embedding size, online KV store, network routing, CPU and GPU clusters
2. dive very deep, check data points from each layer, compare data across different region and time. **Critically evaluate the data points and performance metrics.**
3. found the root cause, there are several layers are bottleneck
4. focus on fixed the bug quick, using 1-2 weeks, tried different approach, fixed the problem.
5. Systematically improve the engine performance, from 1) service layer, routing, 2) decrease model size, 3) using different ways to optimize ML models, 4) using quantization algorithms to decrease memory size, make inference faster.
6. I was the tech lead for the entire project, leading >10 engineers, across 4 different engineering teams, they are not even belong to my director. Across 4 different time zones

results:

1. One to end latency reduced 20%, improved user experience, CTR and day increased 2%.
2. System performance improved a lot, CPU/GPU utilization trigger alarm much less frequency.
3. We are able to train and use bigger model in Redo engine, using existing resources.
4. Saved a lot of cost.

#-----
Dive deep, story 2
#-----

Leading product kernel function accuracy error in HK Research Institute

Situation (1-2 minutes):

1. training model, loss curve some time spike

2. not always reproducible, sometimes spike, sometimes not
3. very bad impression to our customer, losing trust, and cause our customer lots of money, 1 week , one hundred million rmb in china

Task (1-2 minutes):

1. root cause
2. quick fix
3. how to systematically solve this problem

Action (5-6 minutes):

1. collect lots of profiling data, across several layers, e.g., training script, pytorch, parallelization, kernal functions
2. dive very deep, check data points from each layer, compare data with different training epochs
3. found the root cause, vector norm kernal function
the root cause is because the sequence change. theoretically it should not happen.
4. focus on fixed the bug quick, using 1-2 weeks, tried different approach, fixed the problem.
5. create a system, automatically generate all types of unit test cases, to improve code coverage, early finding problems.

results:

1. my LLM training process no spike anymore. training process is fluent.
save our customer lots of money, 100 million / week.
2. shared the root cause, quick fix, with other teams. they have similar problems.
3. broadcast my system to auto generate unit test cases at kernal level, to early finding bugs, to our entire org. found many new bugs.

#-----
Dive deep, story 3
#-----

Example: Diagnosing a Subtle Numerical Instability in an LLM Kernel

Amazon Leadership Principle in Focus: Dive Deep

Leaders operate at all levels, stay connected to the details, audit frequently, and are skeptical when metrics and anecdotes differ. No task is beneath them.

(Situation & Task - 1.5 minutes)

"In my recent role as a Software Architect at the Hong Kong Research Institute, my team was developing a new, high-performance quantization kernel for LLM inference, implementing optimized Duquant and Hadamard transpose algorithms. The goal was a 30% performance gain. During initial testing, the kernel passed all standard unit and integration tests. However, when we began stress testing under production-like, high-throughput conditions, we started seeing sporadic, non-deterministic failures in downstream tasks. The model would occasionally produce gibberish or mathematically incorrect results. The failure was inconsistent and couldn't be easily reproduced, making it a 'heisenbug.'

The initial telemetry and high-level metrics were clean. The team was frustrated, and some suggested it was a hardware flakiness or an issue in the larger framework, not our kernel. My task was to lead the investigation and root-cause this elusive issue, as shipping a numerically unstable kernel was unacceptable for a production-grade inference platform."

(Action - 6 minutes)

"I suspended further rollout and initiated a deep-dive investigation. We considered three primary hypotheses and corresponding investigation strategies.

Option A: Blame the Environment and Strengthen Isolation. Assume the issue is external—either hardware faults or interference from other processes. The approach would be to run the kernel on isolated, dedicated hardware with extensive logging to catch an external culprit.

Option B: Extensive "Shotgun" Logging. Instrument every single function and memory operation in the kernel with verbose logs, then run the stress test again, hoping to capture the failure state and sift through the massive log files for clues.

Option C: A Principled, Layered Deep Dive. Systematically dissect the kernel's operation, from the high-level algorithm down to the individual memory instructions and hardware interactions, using a combination of advanced tools and first-principles reasoning.

Comparison and Trade-off Analysis:

Option A (Blame the Environment):

Pros: Easy to execute. If it works, it passes the problem to another team.

Cons & Risks: This is the opposite of diving deep. It's a defensive, shallow approach. It avoids taking ownership of the kernel's behavior. The likelihood of a clean, isolated environment reproducing a complex concurrency bug is low.

Why I didn't choose it: It was an abdication of our responsibility. Leaders dive deep into their own systems first.

Option B (Shotgun Logging):

Pros: Feels comprehensive. Might get lucky and capture the anomaly.

Cons & Risks: Extremely inefficient. The volume of logs would be overwhelming, and the act of logging itself would likely change the system's timing, potentially hiding the race condition (a Heisenbug effect). It's a brute-force method lacking intellectual rigor.

Why I didn't choose it: It was a reactive, rather than a proactive and analytical, approach. It didn't demonstrate a deep understanding of the system.

Option C (Principled Deep Dive):

Pros: This was the only option that embodied Dive Deep. It involved staying connected to the most minute details of the system. It was a methodical, hypothesis-driven investigation that would lead to a fundamental understanding and a robust fix.

Cons & Risks: Required the most expertise and time. It involved using complex tools like CUDA race condition checkers, memory sanitizers, and manually reviewing assembly-level instructions.

Why I chose this: I was skeptical of the easy answers. To solve a problem at the intersection of algorithms, concurrency, and hardware, we needed to operate at that level. No task—not even reading GPU assembly—was beneath us.

My Decision and Execution:

I led the team through Option C. The deep dive proceeded in layers:

Algorithmic Review: I first reconfirmed the mathematical correctness of the Duquant and Hadamard transpose on paper, ruling out a fundamental flaw.

Code Review: We conducted a line-by-line review of the kernel code, focusing on memory access patterns and synchronization.

Advanced Tooling: We employed NVIDIA's cuda-memcheck and compute-sanitizer tools, which are designed to detect memory access errors and race conditions in GPU code. This was crucial.

Hardware-Level Analysis: When the sanitizers hinted at a potential race, I used the CUDA debugger to inspect the exact sequence of memory operations by different threads within a warp.

The investigation revealed the root cause: a subtle memory corruption due to a race condition. In a specific edge case of the Hadamard transpose, two threads were attempting to write to adjacent memory locations in a way that could cause memory corruption on the hardware memory bus, leading to a few bits in a tensor being flipped. This corruption was small enough to not cause a crash but large enough to completely invalidate the model's calculations under high load.

Backup Plan: The deep dive was intensive. My backup plan was to, in parallel, implement a "safer but slower" version of the kernel that used mutexes for all memory writes. This would have eliminated the race condition at a performance cost. If the root cause analysis exceeded one week, we would have switched to this slower, stable kernel to unblock testing, while continuing the deep dive for the optimal solution."

(Results - 1.5 minutes)

"The deep dive was conclusive. We identified the exact lines of code and the specific memory operations causing the race condition. We fixed it by refactoring the memory access pattern to be inherently safe, without needing heavy-weight synchronization.

The result was a high-performance kernel that delivered the targeted 30% speedup with zero numerical inaccuracies, even under the most extreme stress tests. By diving deep, we didn't just fix a bug; we transformed a flaky, unreliable component into a rock-solid foundation for our

inference platform. This work was directly applicable across different LLM platforms and prevented a critical quality issue from reaching production."

(Feedback - For Self-Improvement)

"While the deep dive was successful, it was triggered reactively by a failure in stress testing. The feedback for myself is to institutionalize 'proactive deep dives' into our development process. Next time, I would advocate for integrating advanced sanitizers and concurrency checkers into our pre-commit testing pipeline for all performance-critical kernels. This would shift our approach from discovering deep issues after they manifest to preventing them from being introduced in the first place, truly baking quality and robustness into the development lifecycle."

#-----
Dive deep, story 3
#-----

Example 1: Debugging Kernel-Level Memory Corruption (Highly relevant for Annapurna)

This story is based on your work at the **Research Institute in Hong Kong** on LLM kernel development.

(S) Situation

As a Software Architect, I was leading the effort to improve our LLM inference platform performance. My team had just developed a new set of quantized kernels, including a custom Hadamard transpose, which were supposed to give us a 30% performance boost. The kernels passed all their individual unit tests, which tested the mathematical logic in isolation.

(T) Task

However, when we integrated these kernels into the full LLM inference pipeline, we hit a critical, non-deterministic bug. About 1 in every 10,000 inference requests would produce calculation inaccuracies—small errors in the final logits. These errors were subtle but unacceptable. The team suspected it was just "quantization noise," but I believed the non-deterministic nature pointed to a deeper, more dangerous systems-level problem. My task was to **Dive Deep** to find the true root cause, as we could not ship a component that produced mathematically incorrect results.

(A) Action

I took **Ownership** of the investigation, knowing that a bug this deep would require going far below the Python layer. I presented three options to my team:

- **Option A: The "Good Enough" Path.**
 - **What:** We could assume this was acceptable quantization noise. We'd add a small tolerance threshold to our end-to-end tests and ship the 30% performance gain.
 - **Trade-off:** This is the fastest path to **Deliver Results** on the performance goal.
 - **Risk:** This violates **Insist on the Highest Standards**. A small, random error in one layer can be amplified across 100+ layers of an LLM, leading to unpredictable, emergent failures. It would destroy trust in our platform. I was not willing to accept this.
- **Option B: The "Containment" Path.**
 - **What:** We could add a "sanity check" wrapper. After the kernel executed, we'd check the output for `NaN` (Not a Number) or `Inf` (Infinity) and, if found, just re-run the kernel for that request.
 - **Trade-off:** This seems safer than Option A and is still relatively fast to implement.
 - **Risk:** This is a bandage, not a fix. It doesn't find the root cause, and the re-run would introduce a latency spike, violating our performance goals for those requests. It just hides the bug.
- **Option C: The "Systems-Level Deep Dive" (My Recommendation).**
 - **What:** I proposed we treat this as a P0 systems bug. My hypothesis was that this was not an *algorithmic* error (which unit tests would catch) but a *memory* error, like a race condition or an out-of-bounds write. I would use low-level debugging tools like `cuda-memcheck` and NVIDIA's `compute-sanitizer` to run the kernel under heavy, parallel load and look for illicit memory access.
 - **Trade-off:** This is the slowest, most difficult path. It requires specialized, low-level skills and could delay the entire project by weeks.
 - **Risk:** The main risk was the time. I could spend a week in a "rabbit hole" and find nothing, putting our performance-gain timeline at risk.

I chose **Option C** because you cannot build a high-performance system on an unstable foundation. We must **Earn Trust** by guaranteeing correctness.

- **Backup Plan:** My backup plan was to time-box the investigation. I gave myself one full week. If I could not find the root cause in that week, we would *revert* this specific kernel, ship the rest of the release with the old, correct-but-slow kernel to unblock other teams, and I would create a dedicated task force to continue the investigation offline. We would not compromise on correctness.

I then spent the next three days running the kernel in a loop with `cuda-memcheck`. I was able to find it: our custom Hadamard transpose implementation had an out-of-bounds memory **write** that occurred *only* when the input matrix dimensions were not a clean multiple of 8. This bug was corrupting a tiny, adjacent piece of memory, which was then used by the *next* kernel in the chain, causing the random calculation errors.

(R) Result

I implemented a fix to correctly handle the memory padding for all matrix sizes. The bug vanished. The kernel was now 100% bit-for-bit accurate.

We successfully shipped the new kernels, achieving the full **30% performance improvement** *without* sacrificing accuracy. More importantly, this **Dive Deep** gave us the confidence to build even more aggressive optimizations on top of this now-stable foundation. We **Earned Trust** with all the modeling teams using our platform.

(Feedback / Learn and Be Curious)

This incident taught me that our standard unit tests were insufficient for kernel development. They only tested "happy path" logic. As a result, I championed a new standard where we integrated these low-level memory-checking tools *directly* into our CI/CD pipeline. We began to "shift left," catching these deep, systemic bugs automatically before they ever reached integration testing.

— — — — —

#technical problems

— — — — —

We want to train a SimpleMLP model on an AIChip which has a peak rating of 100 Tflops.

The memory of each AIChip is 8GB.

The SimpleMLP consists of a stack of 3 layers where

each hidden layer is a linear module followed by a non-linear activation and an output layer in the end

using a simple Mean Square Loss.

Also, for simplification assume that the input and outputs for each of the layer is the same and is a vector of size 10^4 .

what is the total number of parameters of the model?

$\text{hidden_dim} = 10^4$

$\text{num_parm_layer} = \text{hidden_dim}^2 + \text{hidden_dim}$

$\text{num_ttl_parm} = \text{num_parm_layer} * \text{num_layres}$

total memry:

- 1) model parm
- 2) gradints
- 3) optimizer
- 4) activations
- 5) misc

$\text{model_parm} = \text{num_ttl_parm} * 2$

$\text{gradients} = \text{num_ttl_parm} * 2$

$\text{optimizer} = \text{num_ttl_parm} * 2 * 4$ (adam or adamw, fp32)

$\text{activation} = \text{hidden_dim}^2 * \text{batch_size} * \text{num_layers} * 2$

Compute Model Flops Utilization (MFU) of single worker which trains with a throughput of 200 samples/sec.

Also assume that the input is of shape $10^2 \times 10^4$, i.e the sequence length is 10^2 tokens long and

each token has 10^4 as the embedding dimension/size.

```
flops_per_sample = 6 * num_ttl_parms
```

```
actual_flops = flops_per_sample * throughput
```

```
actual_tflops = actual_flops / 1e12
```

```
mfu = (actual_flops / chip_tflops) * 100
```

```
input_dim = seq_len * embedding_dim
```

```
#layer 1
```

```
ttn_parms = (input_dim * hidden_dim) + hidden_dim
```

```
#middle_layer
```

```
ttn_parms += (num_layers - 1) * (hidden_dim^2 + hidden_dim)
```

```
#output layer
```

```
ttn_parms += hidden_dim^2 + hidden_dim
```


How to scale out training for llama 70B:

-

1. adding more hardware

2. software:

1) using CUDA kernel function

2) FA

3) Mlx precision

4) torch.compile

3. parallelism

1) fsdp

2) tp

3) ep

4) sp

4. data pipeline

5. quantization

Training loop that uses FSDP implementation:

```
model.train()
```

```
for epoch in range(epochs):
```

```
    train_loader.sampler.set_epoch(epoch)
```

```
    ttl_loss = 0.0
```

```
    for batch_idx, (data, target) in enumerate(train_loader):
```

```
        data, target = data.to(rank), target.to(rank)
```

```
        optimizer.zero_grad()
```

```
        output = model(data)
```

```
        loss = loss_fn(output, target)
```

```
        ttl_loss += loss.items()
```

```
    loss.backward()
```

```
    optimizer.step()
```

```
    if rank == 0:
```

```
avg_loss = ttl_loss / len(train_loader)

if (epoch % 1000) == 0:
    print("")
```

model =

forward pass:

Assume 4 layer FSDP, 4GPUs

Sharding

layer1 - GPU1

..

layer-n - GPU-n

FWD pass collectives

for layer idx:

all gather model parameters to all other gpus

forward pass

discard model params for layer idx

BWD pass collectives

#using ring all reduce to calculate gradients

#all reduce could be separated reduce and all gather

for layer idx:

all-gather model params

backward pass

discard model params for layer idx

reduce-scatter gradients

to make sure all rank get the updated model parameters

'''

| o | x | |

| x | | |

| x | x | x |

| o | o | |

| o | | |

| x | x | x |

l o l o l l

l o l l l

l o l x l x l

l o l o l l

l o l l l

l o l o l x l

l o l o l l

l o l l l

l o l o l o l

'''

class VirusSpreader:

SICK = 'o'

HEALTHY = 'x'

EMPTY = ''

def __init__(self, room_layout):

#sanity check

```

if not room_layout or room_layout[0]:

    self.grid = []

    self.rows = 0

    self.col = 0

else:

    self.rows = len(room_layout)

    self.col = len(room_layout[0])

    self.grid = [row[:] for row in room_layout]


self.DIRECTIONS = [[-1, 0], [1, 0], [0, -1], [0, 1]]


def _is_valid(self, r, c):

    if r>=0 and r<self.rows and c>=0 and c<self.cols:

        return True

    return False


def calculateSpreadTime(self):

    #sanity check

    if self.rows == 0:

        return 0

    q = deque()

```

```

healthy_count = 0

for r in range(self.rows):
    for c in range(self.cols):
        if self.grid[r][c] == self.HEALTHY:
            healthy_count += 1
        elif self.grid[r][c] == self.SICK:
            q.append((r, c, 0))

if healthy_count == 0:
    return 0

if len(q) == 0:
    return -1

max_mins = 0

while len(q) > 0:
    r, c, time = q.popleft()
    max_mins = time

    for dr, dc in self.DIRECTIONS:
        new_r, new_c = r+dr, c+dc
        if self._is_valid(new_r, new_c) == False:
            continue

```

```
if self.grid[new_r][new_c] != self.HEALTHY:  
    continue
```

```
self.grid[new_r][new_c] = self.SICK  
healthy_count -= 1  
q.append(new_r, new_c, time+1)
```

```
if healthy_count == 0:  
    return max_mins
```

```
return -1
```

'''

5 mins

| o(M) | x |

| o | x(M) |

not spreading

| o(M) | x(M) |

'''

```

import collections

import heapq

from collections import deque

from typing import List, Tuple


class VirusSpreader:
    """
    Simulates the spread of a virus in a room layout using Dijkstra's Algorithm
    to handle variable infection times (weighted edges).

    The room is represented by a 2D grid where seats can be:

    - SICK_NO_MASK ('o'): Spreads to 'x' in 1 minute.
    - SICK_MASKED ('O'): Spreads to 'x' in 4 minutes.
    - HEALTHY_NO_MASK ('x'): Can be infected.
    - HEALTHY_MASKED ('m'): Immune, acts as a barrier.
    - EMPTY (' '): A barrier.
    """

    # --- Constants for seat states ---

    SICK_NO_MASK = 'o'

    SICK_MASKED = 'O'

```

```
HEALTHY_NO_MASK = 'x'
```

```
HEALTHY_MASKED = 'm'
```

```
EMPTY = ''
```

```
# --- Directions for BFS/Dijkstra (Up, Down, Left, Right) ---
```

```
DIRECTIONS = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
def __init__(self, room_layout: List[List[str]]):
```

```
    """
```

```
    Initializes the simulator with a deep copy of the room layout.
```

```
    """
```

```
    if not room_layout or not room_layout[0]:
```

```
        self.grid = []
```

```
        self.rows = 0
```

```
        self.cols = 0
```

```
    else:
```

```
        self.rows = len(room_layout)
```

```
        self.cols = len(room_layout[0])
```

```
        if not all(len(row) == self.cols for row in room_layout):
```

```
            raise ValueError("Grid rows must all be the same length.")
```

```
        self.grid = [row[:] for row in room_layout]
```

```
def _is_valid(self, r: int, c: int) -> bool:
```

```
    """Helper to check if a coordinate is within the grid bounds."""
```

```
    return 0 <= r < self.rows and 0 <= c < self.cols
```

```

def calculate_spread_time(self) -> int:
    """
    Calculates the total time for the virus to infect all reachable
    healthy (no-mask) people using Dijkstra's algorithm.

    Returns:
        int: The number of minutes required to infect the *last*
            reachable 'x' person.
            - Returns 0 if there are no 'x' people to infect.
            - Returns -1 if there are 'x' people but no 'o' or 'O'
              people to start the spread.
            - Returns -1 if there are 'x' people who are unreachable.
    """
    if self.rows == 0:
        return 0

    # Priority Queue for Dijkstra's: stores (time_to_infect, row, col)
    # We use a List and the heapq module.
    pq: List[Tuple[int, int, int]] = []

    # Grid to store the minimum time to infect each cell
    distances = [[float('inf')] * self.cols for _ in range(self.rows)]

    healthy_people_count = 0

```



```

# --- Step 1: Initialize Queue and Count Healthy People ---
for r in range(self.rows):
    for c in range(self.cols):
        state = self.grid[r][c]
        if state == self.HEALTHY_NO_MASK:
            healthy_people_count += 1
        elif state == self.SICK_NO_MASK or state == self.SICK_MASKED:
            # Add all initial virus sources with time 0
            distances[r][c] = 0
            heapq.heappush(pq, (0, r, c))

# --- Step 2: Handle Edge Cases ---
if healthy_people_count == 0:
    return 0 # No healthy people to infect.
if not pq:
    return -1 # Healthy people exist, but no infection source.

# --- Step 3: Run Dijkstra's Algorithm ---
while pq:
    # Pop the person with the *smallest* infection time
    time, r, c = heapq.heappop(pq)

    # If this is a stale entry (we've already found a faster path
    # to this node), skip it.

```

```

if time > distances[r][c]:

    continue

# Determine the spread cost *from* this person
current_state = self.grid[r][c]

if current_state == self.SICK_NO_MASK:

    spread_cost = 1

elif current_state == self.SICK_MASKED:

    spread_cost = 4

elif current_state == self.HEALTHY_NO_MASK:

    # This person was infected at 'time'. They now spread.

    # *** KEY ASSUMPTION ***:

    # A newly infected person becomes SICK_NO_MASK ('o')

    # and spreads with a cost of 1.

    spread_cost = 1

else:

    # This is an EMPTY (' '), HEALTHY_MASKED ('m'),

    # or a stale node. It cannot spread.

    continue

# Explore all 4 adjacent neighbors

for dr, dc in self.DIRECTIONS:

    nr, nc = r + dr, c + dc

```

```

# Check if neighbor is valid AND is an infectable person
if (self._is_valid(nr, nc) and
    self.grid[nr][nc] == self.HEALTHY_NO_MASK):

    new_time = time + spread_cost

# If we found a shorter path to infect this neighbor
if new_time < distances[nr][nc]:
    distances[nr][nc] = new_time
    heapq.heappush(pq, (new_time, nr, nc))

# --- Step 4: Final Check ---
# Find the max time it took to infect any healthy person.
# If any healthy person is still 'inf', they were unreachable.
max_minutes = 0
for r in range(self.rows):
    for c in range(self.cols):
        if self.grid[r][c] == self.HEALTHY_NO_MASK:
            infection_time = distances[r][c]
            if infection_time == float('inf'):
                return -1 # An 'x' was unreachable
            max_minutes = max(max_minutes, infection_time)

return max_minutes

```

```
# --- Test Cases ---
```

```
if __name__ == "__main__":
```

```
    print("--- Running Test Cases (with Masks) ---")
```

```
def run_test(name, layout, expected):
```

```
    print(f"\nTest: {name}")
```

```
    for row in layout:
```

```
        print(f" {row}")
```

```
    try:
```

```
        simulator = VirusSpreader(layout)
```

```
        result = simulator.calculate_spread_time()
```

```
        assert result == expected
```

```
        print(f"Result: {result} (Expected: {expected}) ->  PASS")
```

```
    except Exception as e:
```

```
        print(f"Result: ERROR ({e}) (Expected: {expected}) ->  FAIL")
```

```
# Test 1: Provided Example (but with 'O' SICK_MASKED)
```

```
example_layout_masked = [
```

```
    ['O', 'x', ' '], # 'O' takes 4 mins to infect (0,1) and (1,0)
```

```
    ['x', ' ', ' '],
```

```
    ['x', 'x', 'x']
```

```
]
```

```
# (0,0) infects (0,1) at time 4. (0,1) is now a spreader (cost 1).
```

(0,0) infects (1,0) at time 4. (1,0) is now a spreader (cost 1).

(1,0) infects (2,0) at time $4+1=5$.

(2,0) infects (2,1) at time $5+1=6$.

(2,1) infects (2,2) at time $6+1=7$.

Last infection at time 7.

run_test("Original Example, but SICK_MASKED", example_layout_masked, 7)

Test 2: Healthy Mask ('m') acts as a wall

healthy_mask_wall = [

 ['o', 'm', 'x']

]

'o' cannot infect 'x' because 'm' is a wall.

run_test("Healthy Mask blocks spread", healthy_mask_wall, -1)

Test 3: SICK_MASKED ('O') spreads slowly

slow_spread = [

 ['O', 'x', 'x']

]

(0,0) infects (0,1) at time 4.

(0,1) (now a cost-1 spreader) infects (0,2) at time $4+1=5$.

run_test("SICK_MASKED spreads slowly", slow_spread, 5)

Test 4: Mixed Sources (Dijkstra's main use case)

mixed_source_layout = [

 ['o', 'x', 'x', 'x', 'O']

```

]

# 'o' at (0,0) starts at t=0.

# 'O' at (0,4) starts at t=0.

# 'o' infects (0,1) at t=1.

# (0,1) infects (0,2) at t=1+1=2.

# 'O' infects (0,3) at t=4.

# Path to (0,2) from 'O' is: (0,4)->(0,3) at t=4. (0,3)->(0,2) at t=4+1=5.

# The faster path (t=2) wins.

# Last infection is (0,3) at t=4.

run_test("Mixed Sources (fast and slow)", mixed_source_layout, 4)

```

```

# Test 5: All healthy, no sick

run_test("No Sick People", [['x', 'x'], ['m', 'x']], -1)

```

```

# Test 6: All sick, no healthy

run_test("No Healthy People", [['o', 'O'], ['m', ' ']], 0)

```

```

# Test 7: Original Example (unmasked)

```

```

example_layout_original = [
    ['o', 'x', ' '],
    ['x', ' ', ' '],
    ['x', 'x', 'x']

```

```

]

# (0,0) infects (0,1) and (1,0) at t=1.

# (1,0) infects (2,0) at t=2.

```

```
# (2,0) infects (2,1) at t=3.
```

```
# (2,1) infects (2,2) at t=4.
```

```
run_test("Original Example (SICK_NO_MASK)", example_layout_original, 4)
```

```
#-----
```

```
'''
```

Suppose we want to categorize reviews on Amazon products, and group reviews that are similar.

So, given two product reviews, let's say 'review1' and 'review2', write a function that evaluates if these are similar.

Each review is a single sentence, and is a list of words. We denote each word as 'review[x]'.

We say that 'review1' and 'review2' are similar if 'review1[x]' and 'review2[x]' are synonymous.

In this function, we are given 'review1' and 'review2', as well as a list of synonym pairs 'synonyms'.

```
# Example:
```

```
review1 = ["great", "product"]
```

```
review2 = ["amazing", "device"]
```

```
synonyms = [["great", "amazing"], ["product", "device"]]
```

Result: True

'''

Class Finder:

```
def __init__(self, synonyms):
```

```
    self.parent = {}
```

```
    self.rank = {}
```

```
    for w1, w2 in synonyms:
```

```
        self.union(w1, w2)
```

```
def find(self, word):
```

```
    if word not in self.parent:
```

```
        self.parent[word] = word
```

```
        self.rank[word] = 0
```

```
    return word
```

```
    if self.parent[word] != word:
```

```
        self.parent[word] = self.find(self.parent[word])
```

```
    return self.parent[word]
```

```
def union(self, w1, w2):
```

```
    root1 = self.find(w1)
```

```
    root2 = self.find(w2)
```



```
if root1 != root2:
    if self.rank[root1] < self.rank[root2]:
        self.parent[root1] = root2
        self.rank[root2] += 1
    elif self.rank[root1] > self.rank[root2]:
        self.parent[root2] = root1
        self.rank[root1] += 1
    else:
        self.parent[root2] = root1
        self.rank[root1] += 1

    return True

return False
```

```
def are_synonyms(w1, w2):
    root1 = self.find(w1)
    root2 = self.find(w2)
    if root1 == root2:
        return True

    return False
```

```

class ReviewComparator:

    def __init__(self, synonyms):

        #sanity check

        if not isinstance(synonyms, list):

            raise TypeError("synonyms should be a list")


        self.synonyms_finder = Finder(synonyms)


    def are_review_similar(self, review1, review2):

        if len(review1) != len(review2):

            return False


        for w1, w2 in zip(review1, review2):

            if self.synonyms_finder.are_synonyms(w1, w2) == False:

                return False


        return True

```

```

review1 = ["great", "product"]
review2 = ["amazing", "device"]

```

```
synonyms = [[“great”, “amazing”], [“product”, “device”]]
```

```
#-----
```

Question:

```
/*
```

```
* Question:
```

```
* Implement a cyclic ring buffer of size RING_SIZE. Each element in the ring stores
```

```
* an integer. If the ring is full, newer values should overwrite the oldest
```

```
* values.
```

```
* When reading from the ring, the oldest entry is returned (each entry
```

```
* can be read only once).
```

```
*
```

```
* Implement the data struct and 3 functions below:
```

```
*/
```

```
// [null, null, null]
```

```
// Insert 10, 20, 30
```

```
// [10, 20, 30]
```

```
// Insert 40, 50
```

```
// [40, 50, 30]
```

```
// Read
```

```
// [40, 50, null]
```

```
// Read
```

```
// [null, 50, null]
```

```
#define RING_SIZE 3
```

```
// you don't have to use a struct, it's just here as an example.
```

```
struct my_ring {
```

```
};
```

```
void ring_init(struct my_ring *r)
```

```
{
```

```
}
```

```
int ring_read(struct my_ring *r)
```

```
{
```

```
    return 0;
```

```
}
```

```
void ring_write(struct my_ring *r, int val)
```

```
{
```

```
}
```

```
# -*- coding: utf-8 -*-
```

```
"""
```

Implementation of a Cyclic Ring Buffer in Python.

"""

class CyclicRingBuffer:

"""

A cyclic ring buffer of a fixed size.

This data structure implements a first-in, first-out (FIFO) queue with a fixed capacity. When the buffer is full, newly written values will overwrite the oldest values in the buffer. When reading, the oldest value is returned and removed from the buffer.

All operations (write, read) are performed in constant time.

Attributes:

capacity (int): The maximum number of elements the buffer can hold.

buffer (list): The internal list used to store buffer elements.

head (int): A pointer to the index of the oldest element (next to be read).

tail (int): A pointer to the index of the next available slot for writing.

size (int): The current number of valid elements in the buffer.

Time Complexity:

- Initialization: $O(N)$ where N is the capacity, due to list pre-allocation.
- Write: $O(1)$
- Read: $O(1)$

Space Complexity:

- $O(N)$ where N is the capacity, to store the buffer elements.

```
"""
```

```
def __init__(self, capacity: int):
```

```
    """
```

Initializes the ring buffer with a given capacity.

Args:

capacity: The maximum number of elements the buffer can hold.

Must be a positive integer.

Raises:

ValueError: If capacity is not a positive integer.

```
"""
```

```
if not isinstance(capacity, int) or capacity <= 0:
```

```
    raise ValueError("Capacity must be a positive integer.")
```

```
self.capacity = capacity
```

```
self.buffer = [None] * self.capacity
```

```
self.head = 0
```

```
self.tail = 0
```

```
self.size = 0
```

```
def write(self, value: int) -> None:
```

```
"""
```

Writes an integer value to the buffer.

If the buffer is full, this operation overwrites the oldest value.

Args:

value: The integer value to write into the buffer.

```
"""
```

```
self.buffer[self.tail] = value
```

```
self.tail = (self.tail + 1) % self.capacity
```

```
if self.size == self.capacity:
```

```
    # When full, a write operation overwrites the oldest element (at the head).
```

```
    # Therefore, the new head becomes the next element in the sequence.
```

```
    self.head = (self.head + 1) % self.capacity
```

```
else:
```

```
    # If not full, the number of elements simply increases.
```

```
    self.size += 1
```

```
def read(self) -> int | None:
```

```
"""
```

Reads and removes the oldest element from the buffer.

Each element can be read only once. After being read, its slot in the internal buffer is marked as empty (set to None).

Returns:

The oldest integer value in the buffer, or None if the buffer is empty.

```
"""
```

```
if self.size == 0:
```

```
    return None
```

```
value = self.buffer[self.head]
```

```
# Invalidate the entry after reading to make debugging clearer.
```

```
self.buffer[self.head] = None
```

```
self.head = (self.head + 1) % self.capacity
```

```
self.size -= 1
```

```
return value
```

```
@property
```

```
def is_empty(self) -> bool:
```

```
    """Returns True if the buffer is empty, False otherwise."""
```

```
    return self.size == 0
```

```
@property
```

```
def is_full(self) -> bool:
```

```
    """Returns True if the buffer is full, False otherwise."""
```

```
    return self.size == self.capacity
```

```
def __str__(self) -> str:
```



```
"""Provides a developer-friendly string representation of the buffer's state."""
```

```
items = ', '.join(map(lambda x: str(x) if x is not None else 'null', self.buffer))
```

```
return (f"RingBuffer(capacity={self.capacity}, size={self.size}, "
```

```
    f"head={self.head}, tail={self.tail}, buffer=[{items}])")
```

```
def run_tests():
```

```
    """
```

```
    Runs a comprehensive test suite for the CyclicRingBuffer class.
```

```
    """
```

```
    print("--- Running Test Suite for CyclicRingBuffer ---\n")
```

```
    # Test Case 1: Initialization
```

```
    print("Test 1: Initialization")
```

```
    ring = CyclicRingBuffer(3)
```

```
    assert ring.capacity == 3, "Test 1.1 Failed: Incorrect capacity"
```

```
    assert ring.is_empty, "Test 1.2 Failed: Should be empty"
```

```
    assert not ring.is_full, "Test 1.3 Failed: Should not be full"
```

```
    assert ring.read() is None, "Test 1.4 Failed: Reading from empty should return None"
```

```
    print(f"Initial state: {ring}")
```

```
    print("--> Test 1 PASSED\n")
```

```
    # Test Case 2: Conforms to the prompt's example sequence
```

```
    print("Test 2: Prompt's example sequence")
```

```
    ring = CyclicRingBuffer(3)
```

```
# Insert 10, 20, 30 -> [10, 20, 30]

ring.write(10)

ring.write(20)

ring.write(30)

print(f"After writing 10, 20, 30: {ring}")

assert ring.buffer == [10, 20, 30], "Test 2.1 Failed"

assert ring.is_full, "Test 2.2 Failed: Should be full"
```

```
# Insert 40, 50 -> [40, 50, 30]

ring.write(40) # Overwrites 10

ring.write(50) # Overwrites 20

print(f"After writing 40, 50: {ring}")

assert ring.buffer == [40, 50, 30], "Test 2.3 Failed"
```

```
# Read -> [40, 50, null]

val1 = ring.read() # Should read 30 (oldest)

print(f"Read {val1}, State: {ring}")

assert val1 == 30, "Test 2.4 Failed"

assert ring.buffer == [40, 50, None], "Test 2.5 Failed"
```

```
# Read -> [null, 50, null]

val2 = ring.read() # Should read 40

print(f"Read {val2}, State: {ring}")

assert val2 == 40, "Test 2.6 Failed"

assert ring.buffer == [None, 50, None], "Test 2.7 Failed"
```

```
print("--> Test 2 PASSED\n")
```

```
# Test Case 3: Reading until empty
```

```
print("Test 3: Reading until empty")
```

```
val3 = ring.read() # Should read 50
```

```
print(f"Read {val3}, State: {ring}")
```

```
assert val3 == 50, "Test 3.1 Failed"
```

```
assert ring.is_empty, "Test 3.2 Failed: Should be empty"
```

```
val4 = ring.read()
```

```
print(f"Read from empty buffer: {val4}")
```

```
assert val4 is None, "Test 3.3 Failed"
```

```
print("--> Test 3 PASSED\n")
```

```
# Test Case 4: Interleaved reads and writes
```

```
print("Test 4: Interleaved reads and writes")
```

```
ring = CyclicRingBuffer(4)
```

```
ring.write(1); ring.write(2); ring.write(3)
```

```
assert ring.read() == 1, "Test 4.1 Failed"
```

```
ring.write(4); ring.write(5) # buffer is now [5, 2, 3, 4] and full
```

```
assert ring.read() == 2, "Test 4.2 Failed"
```

```
assert ring.read() == 3, "Test 4.3 Failed"
```

```
assert ring.read() == 4, "Test 4.4 Failed"
```

```
print(f"State before final read: {ring}")
```

```
assert ring.read() == 5, "Test 4.5 Failed"
```

```
assert ring.is_empty, "Test 4.6 Failed"
```

```
print("--> Test 4 PASSED\n")
```

```
# Test Case 5: Invalid Capacity
```

```
print("Test 5: Invalid capacity initialization")
```

```
try:
```

```
    CyclicRingBuffer(0)
```

```
    assert False, "Test 5.1 Failed: Should have raised ValueError"
```

```
except ValueError as e:
```

```
    print(f"Caught expected error: {e}")
```

```
try:
```

```
    CyclicRingBuffer(-5)
```

```
    assert False, "Test 5.2 Failed: Should have raised ValueError"
```

```
except ValueError as e:
```

```
    print(f"Caught expected error: {e}")
```

```
print("--> Test 5 PASSED\n")
```

```
print("--- All Tests Passed Successfully! ---")
```

```
if __name__ == "__main__":
```

```
    run_tests()
```